

# Alder

## Residual Types and Proof-Producing Mathematical Elaboration

A language in which a mathematical step states its intention,  
its type records its obligations, and its elaboration retains the proof.

---

**Central proposal.** Treat the unfinished justification of a mathematical operation as a typed, scoped residual rather than as a hidden elaboration failure. Infer alternative sufficient premise sets in a precisely bounded fragment; discharge the selected route with ordinary Lean evidence; preserve the authored objects, structures, and strength of conclusion throughout.

**Fourth-round increment.** This article specifies an antichain-based residual inference algorithm, proves its finite-fragment properties, develops a realizable affine observation-frame theorem for behavioral synthesis, and works through exact quotients, local differentiation, and a formally specified algebraic root. A small executable propositional frontend accompanies the design.

---

Prepared for Vladimir Reshetnikov

GPT-6 Astra Pro | 6 September 2026

The Python companion was executed. Its parser, finite inference engine, independent derivation checker, and exact-arithmetic tests are real artifacts. The richer mathematical surface and Lean integration are specified, not implemented. Generated Lean files are supplied but were not compiled in this environment. The distinction is maintained throughout.

## Abstract

A mathematician commonly suppresses a proof because its role is routine, not because its content is absent: a denominator is known to be invertible, an identity holds on a neighborhood, a construction preserves a specified invariant, or a finite calculation determines a universal claim within a proved fragment. The preceding brainstorm rounds correctly placed these obligations above Lean’s kernel. Their two round-three syntheses also correctly identified the remaining problem: repeated agreement on refinements and contracts does not determine an inference algorithm or establish a useful authoring interface.

ALDER makes a narrower, operational proposal. A source expression carries a fixed mathematical interpretation, a proved property interface, and a dependent residual telescope. An unfinished derivation denotes a conditional proof, never an unconditional theorem. In a finite grounded Horn fragment, residual alternatives form antichains of missing-premise sets. The resulting algorithm is terminating, proof-producing, and complete for that fragment; its completeness does not extend to arbitrary Lean entailment. A complementary affine observation-frame theorem identifies circumstances in which a finite collection of *realizable* observations decides a universal behavioral equality, including correlated observations and empty element types. The article gives detailed mathematical examples, conservative translations, a bounded grounding design, an implementation boundary, an explicit kernel-extension research agenda, and an evaluation plan that distinguishes new syntax from better libraries and automation.

## Contents

|          |                                                                                    |          |
|----------|------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>What Alder changes, and what it deliberately inherits</b>                       | <b>1</b> |
| 1.1      | The missing abstraction is unfinished justification . . . . .                      | 1        |
| 1.2      | The two round-three reports are the baseline, not raw material to rename . . . . . | 1        |
| 1.3      | Evidence levels used in this article . . . . .                                     | 1        |
| <b>2</b> | <b>What the inspected Lean and Leant sources actually suggest</b>                  | <b>2</b> |
| 2.1      | Local differentiation: the representation mismatch is specific . . . . .           | 2        |
| 2.2      | An algebraic root: repeated guards surround an already good abstraction . . . . .  | 2        |
| 2.3      | Exact quotients and finite tensor families . . . . .                               | 2        |
| 2.4      | Leant already has important acceptance machinery . . . . .                         | 3        |
| 2.5      | Pinned scope of this review . . . . .                                              | 3        |
| <b>3</b> | <b>The mathematical surface: state the move, infer its routine support</b>         | <b>3</b> |
| 3.1      | A small collection of meaning-bearing verbs . . . . .                              | 3        |
| 3.2      | Contracts are mathematical interfaces, not adjectival suggestions . . . . .        | 4        |
| 3.3      | A statement should show its strength before its proof is expanded . . . . .        | 4        |
| 3.4      | Method hints and required methods are different syntax . . . . .                   | 4        |
| <b>4</b> | <b>A type system with proved views and explicit residuals</b>                      | <b>5</b> |
| 4.1      | Three contexts, with one authoritative Lean context . . . . .                      | 5        |
| 4.2      | Proof residuals are not runtime effects . . . . .                                  | 5        |
| 4.3      | A frozen meaning is not the same as a closed term . . . . .                        | 6        |
| 4.4      | Source-level refinement rules . . . . .                                            | 6        |
| 4.5      | Residual composition must respect dependencies . . . . .                           | 6        |
| 4.6      | Publication is a separate transition . . . . .                                     | 7        |
| <b>5</b> | <b>Behavioral types and the distinction between choosing and proving</b>           | <b>7</b> |
| 5.1      | A behavioral arrow has an ordinary dependent interpretation . . . . .              | 7        |
| 5.2      | Variance is implication with the correct direction . . . . .                       | 8        |
| 5.3      | Composition retains the actual intermediate . . . . .                              | 8        |
| 5.4      | Laws of a whole function are first-class predicates . . . . .                      | 8        |
| 5.5      | Requirement transformers are useful, but not automatically weakest . . . . .       | 8        |
| 5.6      | Synthesis needs an acceptance theorem, not just a ranking score . . . . .          | 9        |

|           |                                                                                 |           |
|-----------|---------------------------------------------------------------------------------|-----------|
| <b>6</b>  | <b>Finite residual inference: an algorithm with a stated theorem</b>            | <b>9</b>  |
| 6.1       | The finite boundary . . . . .                                                   | 9         |
| 6.2       | Antichains of alternatives . . . . .                                            | 9         |
| 6.3       | Proof terms are constructed together with supports . . . . .                    | 10        |
| 6.4       | What the theorem does not say . . . . .                                         | 11        |
| 6.5       | A concrete residual trace . . . . .                                             | 11        |
| 6.6       | Method policies must precede destructive minimization . . . . .                 | 11        |
| <b>7</b>  | <b>How the finite rules are obtained</b>                                        | <b>12</b> |
| 7.1       | Grounding is part of the specification . . . . .                                | 12        |
| 7.2       | An explicit combinatorial bound . . . . .                                       | 12        |
| 7.3       | Do not hide synthesis inside matching . . . . .                                 | 12        |
| 7.4       | The automatic fragment has an escape hatch, not a truth oracle . . . . .        | 12        |
| <b>8</b>  | <b>Worked proof: differentiating an identity without losing its region</b>      | <b>13</b> |
| 8.1       | The exact theorem to preserve . . . . .                                         | 13        |
| 8.2       | The human proof and its formal obligations . . . . .                            | 13        |
| 8.3       | Proposed Alder source . . . . .                                                 | 13        |
| 8.4       | Ordinary Lean remains an important control . . . . .                            | 14        |
| <b>9</b>  | <b>Worked arithmetic: exact quotient transport with a satisfiable contract</b>  | <b>14</b> |
| 9.1       | Separate balance, exact division, and inversion . . . . .                       | 14        |
| 9.2       | The Frey coefficient guards . . . . .                                           | 15        |
| 9.3       | Alder’s requested proof, not an invented arithmetic oracle . . . . .            | 15        |
| 9.4       | What the tiny companion does and does not test . . . . .                        | 15        |
| <b>10</b> | <b>Worked hybrid example: a unique formal root and certified approximations</b> | <b>16</b> |
| 10.1      | The source construction and the desired interface . . . . .                     | 16        |
| 10.2      | A concrete instance from the source . . . . .                                   | 16        |
| 10.3      | A residual certificate needs a branch-sensitive bridge . . . . .                | 17        |
| 10.4      | Why nonzero is not enough . . . . .                                             | 17        |
| 10.5      | The complete certificate path . . . . .                                         | 18        |
| 10.6      | Analytic information is a separate upgrade, not a badge on the jet . . . . .    | 18        |
| <b>11</b> | <b>Behavioral synthesis over realizable observation frames</b>                  | <b>19</b> |
| 11.1      | The limitation of comparing raw affine coefficients . . . . .                   | 19        |
| 11.2      | A realizable affine frame . . . . .                                             | 19        |
| 11.3      | Source semantics must commute with the observation model . . . . .              | 20        |
| 11.4      | The list grammar and its normalization theorem . . . . .                        | 20        |
| 11.5      | Three frames, three different correct answers . . . . .                         | 20        |
| 11.6      | Why the coefficient domain matters . . . . .                                    | 21        |
| 11.7      | How this fits Leant without replacing its existing design . . . . .             | 21        |
| <b>12</b> | <b>Relations, scopes, and mathematical proof moves</b>                          | <b>21</b> |
| 12.1      | Transport belongs to the consumer . . . . .                                     | 21        |
| 12.2      | A useful nontrivial upgrade from almost everywhere . . . . .                    | 22        |
| 12.3      | “Without loss of generality” has a typed interface . . . . .                    | 22        |
| 12.4      | Precision is an index, not a descriptive adjective . . . . .                    | 22        |
| <b>13</b> | <b>Definition packages and universal properties</b>                             | <b>23</b> |
| 13.1      | A definition should export an evidence interface . . . . .                      | 23        |
| 13.2      | Bundling is a boundary decision . . . . .                                       | 23        |
| 13.3      | Property learning is monotone; witness selection need not be . . . . .          | 23        |
| <b>14</b> | <b>Implementation architecture and incremental correctness</b>                  | <b>24</b> |
| 14.1      | An explicit pipeline . . . . .                                                  | 24        |
| 14.2      | Evidence-index entry and identity . . . . .                                     | 24        |
| 14.3      | A conservative answer to incremental elaboration . . . . .                      | 24        |
| 14.4      | What happens when simplification rewrites an anchor . . . . .                   | 24        |
| 14.5      | Proof sharing and the cost of elaboration . . . . .                             | 25        |

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| 14.6      | Structure contexts do not erase instance search problems . . . . .        | 25        |
| <b>15</b> | <b>Should Lean’s kernel type system change?</b>                           | <b>25</b> |
| 15.1      | The first extension can be real without being trusted . . . . .           | 25        |
| 15.2      | An explicit-evidence primitive refinement former . . . . .                | 25        |
| 15.3      | What could a primitive improve? . . . . .                                 | 26        |
| 15.4      | Definitional functoriality is a separate, nontrivial experiment . . . . . | 26        |
| 15.5      | Semantic subtyping is not the default conversion relation . . . . .       | 26        |
| 15.6      | Computational witnesses must not disappear into propositions . . . . .    | 26        |
| 15.7      | Recommended decision . . . . .                                            | 27        |
| <b>16</b> | <b>The executed Alder-0 artifact</b>                                      | <b>27</b> |
| 16.1      | The actual grammar . . . . .                                              | 27        |
| 16.2      | The independent derivation checker . . . . .                              | 27        |
| 16.3      | Generated Lean and its explicit boundary . . . . .                        | 28        |
| 16.4      | Executed results and their units . . . . .                                | 28        |
| 16.5      | What has advanced beyond a paper policy . . . . .                         | 29        |
| <b>17</b> | <b>Evaluation that can distinguish a language from better automation</b>  | <b>29</b> |
| 17.1      | The first production slice . . . . .                                      | 29        |
| 17.2      | Matched controls . . . . .                                                | 29        |
| 17.3      | What to measure . . . . .                                                 | 30        |
| 17.4      | Paired mutations, with meaningful positive fixtures . . . . .             | 30        |
| 17.5      | A legitimate stopping outcome . . . . .                                   | 30        |
| <b>18</b> | <b>Relationship to earlier type-theoretic work</b>                        | <b>31</b> |
| <b>19</b> | <b>Conclusion</b>                                                         | <b>31</b> |
| <b>A</b>  | <b>Answers to the two sets of fourth-round questions</b>                  | <b>32</b> |
| <b>B</b>  | <b>Package contents and reproduction</b>                                  | <b>33</b> |

# 1 What Alder changes, and what it deliberately inherits

## 1.1 The missing abstraction is unfinished justification

Consider the instruction “differentiate the identity.” There are several distinct questions behind it. Which identity? On what region? At which points? Are the two expressions actual differentiable functions, or merely values of Lean’s total derivative operator? Which induction variables must remain generalized? A useful language should infer the routine answers from evidence already available, but must not answer a different question when inference fails.

ALDER represents a step as three related objects:

fixed mathematical meaning + a proof-producing operation + its remaining obligations.

The first is not negotiable during proof search. The second may be chosen by an author, a tactic, a synthesis engine, or a certified computer-algebra service. The third is visible and typed. Concision comes from not writing the routine derivation of an obligation repeatedly, rather than from dropping the obligation or strengthening the advertised conclusion.

This is an extension of the *source typing discipline*. It need not initially be an extension of Lean’s trusted type theory. A user can write a property-implicit signature, and the elaborator can produce an ordinary dependent function with explicit proof arguments. The important addition is the operational contract: how those arguments are discovered, what happens when discovery is incomplete, and what mathematical information is kept fixed.

## 1.2 The two round-three reports are the baseline, not raw material to rename

The two reports *From Properties to Proof Obligations* and *Nine Refinements* converge on same-object property evidence, explicit structure selection, behavioral contracts, theorem-backed computation, and conservative Lean elaboration. They also converge on the absence of a real property-aware authoring interface and a usability measurement. Both now incorporate corrections concerning finite evidence, realizability, finite/free tensor factors, proof counts, and the distinction between supplied hypotheses and checked implementations [1, 2].

ALDER adopts that corrected baseline. In particular, it does not claim that ordinary Lean lacks the logical expressiveness to state the intended properties. Nor does it call a new spelling of a subtype a new foundation. The incremental contributions relative to that baseline are:

1. A residual typing discipline with an explicit distinction between conditional derivations, completed proofs, and authorized witness construction.
2. An executable antichain algorithm for alternative sufficient premise sets, with a stated finite grounding boundary and a correctness argument that covers the actual fixed-point computation.
3. A finite affine observation-frame theorem that handles correlated and restricted observations without demanding equality of raw coefficient vectors.
4. A concrete source-to-derivation-to-Lean-text experiment, plus a new worked formal-root example connecting properties, behavioral constraints, and certified algebra.

These are design and mathematical contributions to this campaign, not claims of historical priority for abduction, refinement types, fixed-point provenance, or affine interpolation.

## 1.3 Evidence levels used in this article

A *source observation* is a fact established by reading a named repository file at a recorded revision. A *paper theorem* has a mathematical proof here but is not represented as newly kernel-checked. An *executed result* is a result of the supplied Python programs. A *proposed feature* is a specification for a future Lean extension. Historical kernel checks performed by the round-three reviewers remain their results, not fresh checks by this article.

The supplied executable, called ALDER-0, handles opaque propositions and finite Horn rules. It parses authored commands, computes residual alternatives, checks derivations with a separate checker, and emits ordinary Lean theorem text whose rule assumptions are explicit. It does not parse arbitrary Lean expressions, validate mathematical theorem registrations, implement dependent unification, or establish that a displayed arithmetic atom has its intended interpretation. This limited artifact tests a real portion of the proposed mechanism without pretending to close the larger integration problem.

## 2 What the inspected Lean and Leant sources actually suggest

### 2.1 Local differentiation: the representation mismatch is specific

The ProveIt theorem `AutonomousODE.iteratedDeriv_eq_comp` proves an identity for all iterated derivatives of a solution of an autonomous differential equation. Its induction hypothesis is valid throughout an open set. To differentiate that identity at a point, the proof converts the on-set identity into eventual equality in the neighborhood filter, transports the derivative, and applies the chain rule [3].

That code is not simply “too long.” It records genuine mathematical distinctions. The desired compression is a reusable rule for moving from an identity on an open region to the corresponding local differential statement, with the open-region evidence retained. It would be a regression to replace the original hypotheses by global differentiability of auxiliary functions: the original theorem deliberately needs them only on the image of the solution over the region.

### 2.2 An algebraic root: repeated guards surround an already good abstraction

`AlgebraicInverseGerm.lean` builds a polynomial over a power-series ring from a jet and correction weights. The generic formal-root engine already exists. Its application requires a vanishing constant coefficient and an invertible linear coefficient; the file proves the coefficient-reduction bridge, invokes existence and uniqueness, defines a distinguished noncomputable root, and exports its equations [4].

The opportunity is not to rediscover formal implicit-function theory through search. It is to expose a single branch-sensitive interface, infer the coefficient guards through the proved reduction bridge, and reuse the resulting root properties without repeating proof parameters. The data “the chosen zero-constant root” and the theorem “it is unique among zero-constant roots” have different roles. A finite jet of this root is a third object, related to it only to a stated precision.

### 2.3 Exact quotients and finite tensor families

The Fermat repository’s `S_FreyPackage_freyCurveInt_map.lean` contains a coefficientwise proof that an integral Weierstrass model maps to the rational one. The substantial steps establish divisibility before commuting integer division with the cast. Its terminal exported solution is already a short wrapper [5]. A new surface should compress the repetitive guard assembly, not obscure the distinction between integer quotient and field division.

The inspected finite tensor-family theorem explicitly assumes that every factor is finite and free as a module. Its proof constructs representing algebras by finite-type induction and composes equivalences, then verifies naturality. The assumptions are present in the source; their omission in one earlier proposal was an expository mistake, correctly identified by the syntheses [6, 1, 2]. This is an especially useful test: requesting a finite/free output should expose the missing factor guarantees, whereas the universal-property part can remain meaningful without them.

## 2.4 Leant already has important acceptance machinery

At the directly reviewed Leant revision, **Verified** is explicitly an opaque callback-acceptance receipt, not a stored kernel proof. Verification preserves the exact accepted candidate and makes success quotas depend on accepted candidate groups. Post-verification ordering uses generative epochs and nominal types to prevent reassociation across batches; a seal validates that the proposed ordering is an exact permutation of the retained occurrences [7, 8].

The Length contract vocabulary already distinguishes exact spine identities, provider argument roles, provider laws, candidate-case authority, and *joint two-output* contracts. Its comments describe these values as passive assertions until the handoff checks their association with the translated candidate. They are not semantic theorems about arbitrary provider implementations [9].

ALDER should preserve these boundaries and attach proof-linked behavioral evidence to them. It should not replace this infrastructure with a vague “ask an AI to synthesize a term” box. The syntheses also describe a later Leant snapshot with positive and negative **decide** checks of exact candidate propositions. That description is treated here as their source finding, not silently attributed to the public-main revision directly inspected for this article [1, 2].

## 2.5 Pinned scope of this review

| Repository        | Revision prefix | Directly inspected material                                                             |
|-------------------|-----------------|-----------------------------------------------------------------------------------------|
| Brainstorm        | bf3333905995    | Both round-three main reports, the peer appendix, and the synthesis crosswalk.          |
| ProveIt           | 24ce8bd743ea    | Autonomous iterated derivatives and the algebraic inverse germ.                         |
| Fermat repository | aa2d8b34692b    | Integral-to-rational Frey model transport and finite/free tensor-family representation. |
| Leant             | 3a40904be8a4    | Verification, post-verification sealing, and scalar/joint Length contract vocabulary.   |

Full revisions and paths are recorded in the bibliography and `evidence/source_register.json`. This is a focused source review, not a build or audit of any of these repositories. The individual round-three proposals are discussed through the two syntheses; no claim is made to have independently reread all nine original articles.

# 3 The mathematical surface: state the move, infer its routine support

## 3.1 A small collection of meaning-bearing verbs

The proposed mathematical surface distinguishes establishing evidence from selecting data. **Fix** introduces a parameter. **Assume** introduces a visible hypothesis. **Know** proves a property of an existing object. **Construct** creates a new witness under a fixed specification. **Use** applies a specified mathematical rule. **Calculate** requests an equality or another explicitly named relation. **Explain** reports the residual of an unfinished operation without asserting its result.

For example, the following is proposed ALDER syntax, not the grammar of the shipped prototype:

```

fix R : CommRing
fix J : Polynomial R where coeff J 0 = 0, IsUnit (coeff J 1)
fix b : Nat -> R where b 0 = 1

construct S : PowerSeries R
  uniquely where constantCoeff S = 0 and germPolynomial b J eval S = 0
  using formal_implicit_root

know constantCoeff S = 0
calculate germPolynomial b J eval S = 0

```

Here `usingformal_implicit_root` denotes a project-owned registration linked to the actual theorem signature, not a theorem inferred from its English name. The last two lines can be immediate projections from the accepted construction. The existence proof may use classical choice for the distinguished mathematical object; this does not promise an executable program that computes an infinite power series.

### 3.2 Contracts are mathematical interfaces, not adjectival suggestions

A sorting request illustrates what the surface must preserve:

```

construct sort : List A -> List A
  where forall xs,
    Permutation xs (sort xs) and Sorted le (sort xs)
  search providers [merge, split, singleton, empty]

```

The carrier type alone is inadequate; length alone is inadequate; sortedness alone is inadequate. The actual input occurs in the permutation condition. The order relation is fixed before search. If stability is intended, it is an additional relation on the positions of equal-key elements, not an automatic consequence of sorting.

A separate `executable` mode would require a computational implementation and an explicit operational semantics for any cost or termination claim. A mathematically total Lean function, an executable implementation, and a proof about its running time are not interchangeable artifacts.

### 3.3 A statement should show its strength before its proof is expanded

The compact view must retain the data that can change the truth conditions: domain, scalar structure, measure, local region, branch, quantifier order, precision, and whether the result is a witness, a complete solution set, a bound, or a least bound. Routine derivations may be folded. Those distinctions may not.

A calculation has a typed relation:

$$a = b, \quad f =^{\text{a.e.}[\mu]} g, \quad S \equiv T \pmod{Q^N}, \quad |x - y| \leq \varepsilon.$$

There is no global ranking in which each is merely a stronger or weaker badge. A consumer theorem determines which relation suffices. An interval enclosure with equal endpoints can establish equality; an arbitrary enclosure cannot. A jet relation supports finite-order algebra but does not identify an entire series.

### 3.4 Method hints and required methods are different syntax

A reason supplied as a hint may be replaced by another successful route. A required method is an explicit constraint on a derivation language or on the permitted helper theorem interface. ALDER distinguishes them:



```
show G hint chain_rule
show G by required local_differentiation
```

The second form does not promise to recover a human’s intent from an arbitrary Lean proof term. It requests a derivation assembled through a specified checked rule interface. For a support-sensitive benchmark, compiling a helper with only the admitted premises is often simpler and more reliable than searching a huge proof term for forbidden dependencies.

## 4 A type system with proved views and explicit residuals

### 4.1 Three contexts, with one authoritative Lean context

Let  $\Gamma$  be an ordinary Lean dependent context, including parameters and local hypotheses. Let  $\Sigma$  record the already selected mathematical structures and the interpretation of the authored target. Let  $E$  be an index of available proofs in  $\Gamma$ . The index is not a second logical context: its entries must be actual proof terms whose types Lean can check in  $\Gamma$ .

A completed property view can be written

$$\Gamma; \Sigma; E \vdash t : A \triangleright \rho,$$

where  $\rho$  is a finite telescope or conjunction of propositions about the exact elaborated object  $t$ . For example,  $\rho$  may contain  $\text{IsUnit}(a)$  or  $\forall x \in U, \text{HasDerivAt}(f, d(x), x)$ . The carrier remains  $A$  and the object remains  $t$ .

An unfinished proof-producing step additionally returns a residual:

$$\Gamma; \Sigma; E \vdash s \rightsquigarrow (p : T) ! \Delta. \quad (1)$$

Here  $\Delta = (d_1 : D_1, \dots, d_k : D_k)$  is a well-formed dependent telescope. In ordinary proof mode its declarations are proof obligations,  $D_i : \mathbf{Prop}$ ; separately authorized construction requests can also bind data witnesses, whose role is recorded explicitly. The meaning of the judgment is

$$\Gamma, \Delta \vdash p : T, \quad \text{hence} \quad \Gamma \vdash \lambda \Delta. p : \Pi \Delta. T. \quad (2)$$

When  $T$  is independent of the residual proofs this is an ordinary conditional theorem. In the general dependent case the telescope, its order, and the dependencies of  $T$  must be retained. A flat unordered set of printed obligations is only an editor projection of this representation.

### 4.2 Proof residuals are not runtime effects

The exclamation mark in (1) is an *elaboration effect*: the step demands justification before publication. It is not an exception thrown while evaluating the mathematical function. For example, a totalized Lean division remains totalized when imported. A guarded theorem about cancellation can have a residual nonzero premise even though evaluating the division term itself requires no proof.

There are three distinct situations:

| Interface                                    | Meaning                                                                          |
|----------------------------------------------|----------------------------------------------------------------------------------|
| $t : A$ with a residual proof of $P(t)$      | The data exists already; a requested property remains unproved.                  |
| $f : (x : A) \rightarrow P(x) \rightarrow B$ | Applying this interface really requires a proof argument.                        |
| $f : \{x : A \mid P(x)\} \rightarrow B$      | The function is defined on a refined domain; no extension to all $A$ is implied. |

Conflating the three would produce either unnecessary restrictions or unsound extensions. **ALDER** records the imported function’s actual interface before deciding which proof work is implicit.

### 4.3 A frozen meaning is not the same as a closed term

A theorem statement may still contain local variables; a construction may still contain an authorized witness hole. “Frozen” means that proof search cannot instantiate the holes that determine what the author has asked to prove. The selected ring operations, region, measure, branch predicate, and universal quantifier structure belong to that meaning.

Metavariables receive roles from the elaborated request, not merely from whether their types live in **Prop** or **Type**. A proposition metavariable can change the statement. A data metavariable can be a harmless authorized witness. Proof-valued indices can occur in dependent types. A usable implementation must inspect these dependencies and reject an assignment that escapes its role, rather than relying on a syntactic ban on all data holes.

A construction request is therefore sealed as

$$\text{Request}(A, \text{Spec}) = \text{“find } a : A \text{ and } h : \text{Spec}(a)\text{”}.$$

Search may choose  $a$  because that choice was delegated. It may not weaken  $\text{Spec}$  or change  $A$  in order to accept the candidate.

### 4.4 Source-level refinement rules

For a fixed carrier  $A$ , the basic rules are ordinary evidence assembly:

$$\frac{t : A \quad h : P(t)}{t : A \triangleright P(t)} \quad \frac{h : P(t) \quad k : Q(t)}{\langle h, k \rangle : P(t) \wedge Q(t)}.$$

Subsumption is proof insertion, not conversion:

$$\frac{h : P(t) \quad r : \forall x : A, P(x) \rightarrow Q(x)}{r \ t \ h : Q(t)}.$$

At an API boundary, a view can be packaged as  $\langle t, h \rangle : \{x : A \mid P(x)\}$ . Locally, repeated repackaging is avoided. This keeps existing dependent expressions anchored to the same  $t$ .

An equality  $e : t = u$  permits transport of  $h : P(t)$  to  $P(u)$  by equality elimination. A proof  $P(t) \leftrightarrow Q(u)$  instead supplies a proposition-level adapter. Neither licenses the claim that two unrelated structure dictionaries or witnesses are identical.

### 4.5 Residual composition must respect dependencies

Suppose the first stage constructs  $p$  under  $\Delta_1$ , and the next stage uses  $p$  while generating  $\Delta_2(p)$ . Their composition has the telescope

$$\Delta_1, \Delta_2(p),$$

not two independently computed, freely permutable bags of premises. If a residual mentions a witness introduced in an existential elimination, it cannot escape that witness’s scope. Export must either close the existential correctly, generalize the entire dependent telescope when justified, or keep the work local.

For the finite inference algorithm below, all meaning-bearing data and dependent indices have already been elaborated. Its atoms are fixed well-typed propositions in a fixed context. The set-based support calculus is justified at that boundary; it is not a general replacement for dependent telescopes.

**Proposition 4.1** (Conditional elaboration and completion). *Assume every rule used by a residual derivation is an ordinary Lean theorem application or a valid Lean structural rule, and the translation preserves its typed context. Then a derivation of (1) produces the conditional term in (2). If  $\Gamma$  supplies a well-typed substitution  $\delta$  for every declaration of  $\Delta$ , then  $p[\delta]$  proves  $T[\delta]$  in  $\Gamma$ .*

*Proof.* Induct on the derivation. A known fact is its retained proof. An unresolved proof argument becomes a binder in the residual telescope. Rule application applies the translated theorem to translated arguments, preserving dependent order. Refinement weakening is implication elimination; equality transport is equality elimination; local binding and branch discharge use their ordinary dependent rules. Abstraction over the remaining telescope gives the conditional term. Typed substitution yields the completed term.  $\square$

This proposition states the intended translation invariant. It does not verify a future parser or metavariable implementation. The finite algorithm receives a more concrete correctness argument in Section 6, and the supplied parser has executed tests described in Section 16.

## 4.6 Publication is a separate transition

A `show` command succeeds only when its exact requested assertion has a completed proof. An `explain` command may return a conditional derivation and its alternatives. An explicit `export conditional` operation may publish the conditional theorem, showing its premises in the statement. The system must never turn an unresolved guard into an unnoticed theorem hypothesis.

All displayed mathematical assertions are checked, not just the statements on which the terminal theorem depends. This requirement is stricter than “the file contains a proof of its final theorem.” A valid final proof can coexist with an unused false sentence; a mathematical document cannot label that sentence checked merely because it was unused.

# 5 Behavioral types and the distinction between choosing and proving

## 5.1 A behavioral arrow has an ordinary dependent interpretation

For fixed  $f : A \rightarrow B$ , write

$$\text{Beh}(P, Q, f) := \forall x : A, P(x) \rightarrow Q(x, f(x)).$$

A source type

$$A \xrightarrow{P; Q} B$$

can be packaged at a boundary as  $\{f : A \rightarrow B \mid \text{Beh}(P, Q, f)\}$ . Internally,  $f$  and its behavioral proof can remain separate, with the proof indexed under the exact function term. The domain  $A$ , the precondition  $P$ , and the input-output relation  $Q$  are fixed before candidate search.

A source-level property-implicit binder might be written

```
def cancel (a b c : R) requires IsRegular a :
  a * b = a * c -> b = c
```

Its conservative translation introduces a regularity proof argument at the correct position. It does not put regularity into definitional equality, and does not automatically infer that every nonzero element of the chosen ring is regular.

## 5.2 Variance is implication with the correct direction

Suppose  $f$  has contract  $(P, Q)$ . It may be used with a stronger precondition  $P'$  and a weaker postcondition  $Q'$  when there are proofs

$$P'(x) \rightarrow P(x), \quad P'(x) \rightarrow Q(x, y) \rightarrow Q'(x, y).$$

Then  $P'(x)$  supplies the original precondition, the original contract gives  $Q(x, f(x))$ , and the second implication supplies  $Q'(x, f(x))$ . This is contravariance in the precondition and covariance in the guarantee, not a numerical ordering of contracts by “strength.”

For a refined-domain function the variance problem also includes a map between domains. A function on  $\{x \mid P(x)\}$  can restrict to  $\{x \mid P'(x)\}$  using  $P' \rightarrow P$ . Extending it to a larger domain requires new data or additional structure. It is not behavioral weakening.

## 5.3 Composition retains the actual intermediate

Let  $f : A \rightarrow B$ ,  $g : B \rightarrow C$  and assume

$$\begin{aligned} H_f &: \forall x, P(x) \rightarrow Q(x, f(x)), \\ H_g &: \forall y, R(y) \rightarrow S(y, g(y)), \\ B_{fg} &: \forall x y, P(x) \rightarrow Q(x, y) \rightarrow R(y). \end{aligned}$$

The bridge  $B_{fg}$  is the precondition obligation generated when the functions are composed. The resulting contract is

$$\forall x, P(x) \rightarrow (Q(x, f(x)) \wedge S(f(x), g(f(x)))). \quad (3)$$

The intermediate is not an independently chosen  $y$  satisfying a similar predicate: it is the actual  $f(x)$ . Existentially hiding it gives a relational composition contract, but hiding it must not discard correlations used later.

When  $g(x, y)$  has a result type depending on  $y$ , the natural composite may return a dependent pair  $\langle y, z \rangle$  rather than an element of a fixed  $C$ . A source presentation may hide the pair only when its consumer has the corresponding dependent interface. This is one place where an untyped “postcondition row” is inadequate.

## 5.4 Laws of a whole function are first-class predicates

Monotonicity, injectivity, Lipschitz bounds, commutation, naturality, preservation of an ideal, and stability of a sort are properties of one or more whole functions. They are not all single-run postconditions. ALDER permits arbitrary ordinary Lean predicates in its property interface, while giving automatic support only to registered fragments.

For example, a natural family of equivalences must satisfy a square involving both objects and every admissible morphism. Proving each component bijective does not prove that square. Similarly, two lists produced together may satisfy a joint length equation without either output independently satisfying a fixed scalar length contract. The representation of a law should preserve the tuple on which the law depends.

## 5.5 Requirement transformers are useful, but not automatically weakest

A registered method  $M$  can have a proved sufficient requirement transformer

$$\text{Req}_M(\text{requested guarantee}).$$

For a deterministic pure function  $f$ , the semantic weakest precondition for a postcondition  $Q$  is simply  $x \mapsto Q(f(x))$ . Useful library contracts often provide stronger, more tractable requirements.

The residual engine combines those registered sufficient routes. It does not thereby compute the semantic weakest precondition of an arbitrary mathematical operation.

For actual monadic program verification, Lean’s `mvengen` already interprets programs through predicate transformers, uses specification lemmas, and generates verification conditions. It is a baseline and integration point, not a capability invented by ALDER [12]. The source-level residual mechanism can present its obligations alongside mathematical ones while preserving the distinction between runtime state semantics and unfinished proof work.

## 5.6 Synthesis needs an acceptance theorem, not just a ranking score

A candidate producer is untrusted. The accepted artifact must establish

$$\Gamma \vdash c : A, \quad \Gamma \vdash h : \text{Spec}(c)$$

for the exact accepted candidate and the exact frozen specification. A score, test outcome, callback receipt, or model-relative certificate is not a substitute. It may guide search, but the final bridge must connect it to  $\text{Spec}(c)$ .

A counterexample to one completed candidate refutes that candidate’s universal contract. It does not refute the existence of a different implementation. Rejecting a partial sketch requires a theorem covering all its permitted completions, or a clearly heuristic search decision that carries no such logical conclusion. These distinctions are inherited from the corrected round-three analyses and become explicit result constructors in the proposed API.

## 6 Finite residual inference: an algorithm with a stated theorem

### 6.1 The finite boundary

After grounding, let  $\mathcal{Q}$  be a finite set of well-typed propositions in a fixed context. Let  $\mathcal{R}$  be a finite set of Horn rules

$$r : q_1 \rightarrow \cdots \rightarrow q_k \rightarrow q, \quad q_i, q \in \mathcal{Q},$$

each backed by a theorem at that exact instantiated type. Let  $K \subseteq \mathcal{Q}$  be known propositions with retained proofs. Let  $M \subseteq \mathcal{Q}$  be the explicitly permitted *repair vocabulary*: propositions whose proofs the user is willing to supply or delegate as residuals.

The repair vocabulary is not automatically all atoms. In particular, making the desired conclusion itself a permitted repair yields the tautological explanation “the conclusion follows if you prove the conclusion.” That can be logically valid and editorially useless. The default repair vocabulary consists of input qualifications and selected operation guards, not arbitrary conclusions.

A support  $S \subseteq M$  means that  $K$  together with proofs of the propositions in  $S$  suffices to derive the target using  $\mathcal{R}$ . It does not assert that any member of  $S$  is true. Nor does it assert that the support is satisfiable jointly with  $K$ ; a claimed constructive repair may need a separate feasibility check. The benchmark’s intended positive fixtures should establish that feasibility rather than merely exploit a contradictory context.

### 6.2 Antichains of alternatives

For a family  $X \subseteq \mathcal{P}(M)$  define

$$\text{Min}(X) = \{S \in X : \text{there is no } T \in X \text{ with } T \subsetneq S\}.$$

Store for each  $q$  an antichain  $F(q)$  of inclusion-minimal supports, each with one derivation. The operations are

$$X \oplus Y = \text{Min}(X \cup Y), \quad (4)$$

$$X \otimes Y = \text{Min}\{S \cup T : S \in X, T \in Y\}. \quad (5)$$

Alternative proof routes use  $\oplus$ ; simultaneous premises use  $\otimes$ . The empty family means no route has been found in this calculus. The singleton family  $\{\emptyset\}$  means a completed route is available. These are opposite states, despite both involving an empty object.

Initialize  $F(q)$  with  $\emptyset$  if  $q \in K$ , and with  $\{q\}$  if  $q \in M$ . For each rule  $q_1, \dots, q_k \Rightarrow q$ , repeatedly perform

$$F(q) \leftarrow F(q) \oplus \bigotimes_{i=1}^k F(q_i). \quad (6)$$

An empty-premise rule contributes  $\{\emptyset\}$ ; in a real registry it must be a theorem with no unfulfilled proof premises, not a fact inferred from a label. In ALDER-0 it is an explicit theorem parameter in generated Lean.

### 6.3 Proof terms are constructed together with supports

A known leaf is its actual retained proof. A missing leaf  $m$  is a new local parameter  $h_m : m$  in the conditional derivation. At a rule node, apply the registered theorem to the child derivations in its declared order. The support annotation is the union of the child supports.

When a smaller support supersedes a larger one, removing the larger support from the search index does not invalidate proofs already referring to its derivation. Derivation nodes are immutable. The search index is a set of preferred routes; it is not the ownership boundary of the proof DAG.

The supplied algorithm keeps the first derivation encountered for equal supports. It does not find the globally shortest proof, the cheapest mathematical route, or a minimal set of theorem dependencies. These are distinct optimization problems. A later implementation may retain a bounded Pareto set of costs, but that is not needed for the semantic claim below.

**Theorem 6.1** (Termination and exact residual alternatives). *For finite  $\mathcal{Q}$ ,  $\mathcal{R}$ ,  $K$ , and  $M$ , complete fair iteration of (6) terminates. At its fixed point:*

1. *Every  $S \in F(q)$  has a derivation of  $q$  from  $K \cup S$  using  $\mathcal{R}$ .*
2. *For every  $S \subseteq M$ ,  $q \in \text{cl}_{\mathcal{R}}(K \cup S)$  if and only if some  $T \in F(q)$  satisfies  $T \subseteq S$ .*
3.  *$F(q)$  is precisely the family of inclusion-minimal supports admitting such a derivation. It is independent of the fair update schedule, although the chosen proof DAG need not be.*

*A completed proof is available exactly when  $\emptyset \in F(q)$ .*

*Proof.* Interpret an antichain by its upward closure

$$\uparrow F(q) = \{S \subseteq M : \exists T \in F(q), T \subseteq S\}.$$

Every genuinely new insertion enlarges this set; deleting supersets during minimization does not shrink it. There are only  $|\mathcal{Q}|2^{|M|}$  atom-support pairs. Thus only finitely many successful insertions are possible. Iteration stops when a full rule pass makes none.

Soundness is induction on node creation. The two kinds of leaf have the announced support. At a rule application, all premises follow from the union of the children's supports, so its theorem establishes the conclusion. Immutable children predate their parent; no unsupported cyclic proof can be created.

For completeness, take a finite Horn derivation from  $K \cup S$  and induct on its height. A known leaf has support  $\emptyset \subseteq S$ . A leaf supplied from  $S$  has a seed support contained in  $S$ , possibly already

dominated by a smaller one. For an internal rule, the induction hypotheses give stored supports  $T_i \subseteq S$  for its premises. At the fixed point their union, or a smaller support that dominates it, is represented for the conclusion. Hence some  $T \in F(q)$  satisfies  $T \subseteq S$ .

The reverse implication in item 2 follows from soundness and weakening. Minimality then identifies  $F(q)$  uniquely with the minimal elements of the set of all derivable supports. This proves schedule independence at the support level. Taking  $S = \emptyset$  gives the final assertion.  $\square$

## 6.4 What the theorem does not say

The theorem is about derivability in the fixed finite rule system. It is not a decision procedure for all propositions of Lean, a claim that its supports are logically weakest assumptions, or a proof that an absent route is impossible in another library. If the registry omits a direct theorem, the residual engine can miss an already easy proof.

Nor does the theorem make the procedure polynomial. An antichain over  $m$  repair atoms can have  $\binom{m}{\lfloor m/2 \rfloor}$  members. A naive implementation may enumerate products of premise antichains, and proof terms can become large without sharing. The finite bound proves termination of the unbudgeted procedure, not acceptable interactive latency. Bounded execution must report `unknown: budget exhausted`; it must not relabel the unfinished fixed point as a complete explanation.

## 6.5 A concrete residual trace

Consider the abstract rule family

$$\begin{aligned} \text{positive} &\Rightarrow \text{nonzero}, \\ \text{unit} &\Rightarrow \text{nonzero}, \\ \text{exact} &\Rightarrow \text{balanced}, \\ \text{balanced} \wedge \text{nonzero} &\Rightarrow \text{cast}. \end{aligned}$$

Given  $K = \{\text{exact}\}$  and  $M = \{\text{positive}, \text{unit}\}$ , the result is

$$F(\text{cast}) = \{\{\text{positive}\}, \{\text{unit}\}\}.$$

The two conditional derivations are different sufficient routes. Neither establishes that positivity or being a unit is necessary for a cast identity in every ring. The example's atoms are deliberately opaque in the executable: instantiating them to actual mathematical propositions requires a validated theorem registry.

Within a lexical block that assumes positivity, the empty support becomes available. Leaving the block removes that assumption and returns the parent to the two conditional alternatives. This behavior is exercised by the supplied `quotient.alder` example, rather than merely described by pseudocode.

## 6.6 Method policies must precede destructive minimization

If one proof of support  $S$  uses a forbidden method and another proof of the same support does not, keeping only the first and filtering afterward loses a valid route. A simple support-restricted policy therefore filters the registry and admissible known facts *before* inference.

A positive requirement such as “this route must contain the designated local-differentiation step” is different. One principled extension is to annotate derivations with a finite tree-automaton state, compute alternatives for pairs  $(q, \text{state})$ , and accept only the permitted terminal states. This is a proposed extension, not implemented in ALDER-0. It checks a defined derivational property, not a person's psychological reasoning process.



## 7 How the finite rules are obtained

### 7.1 Grounding is part of the specification

The third-round closure theorem assumed a finite ground rule set; the syntheses explicitly asked what generates it [1, 2]. ALDER uses a deliberately restrictive first implementation.

A sealed method application produces a finite typed term arena  $\mathcal{T}$ . It includes the request’s elaborated subterms, the relevant local parameters, and a finite set of intermediate terms explicitly produced by that method’s elaboration template. Every term is accompanied by its local context and its selected structure arguments. A subterm under a binder is not moved into an unrelated context by stripping its binder.

The initial registry admits first-order theorem templates whose data parameters can be assigned from this arena, whose universe instantiations are fixed by the request or an explicit finite set, and whose remaining premises are propositions. Matching the conclusion against demanded atoms often determines most assignments. All remaining assignments are enumerated from the appropriately typed portion of  $\mathcal{T}$ . Every proposed instance is checked against the actual theorem type before it becomes a rule.

No rule may manufacture a fresh data witness during this closure. No result of arbitrary term synthesis is automatically added to  $\mathcal{T}$  in the middle of the fixed point. A successful construction opens a new stage with a new sealed arena. This restriction sacrifices completeness but makes the inference claim meaningful.

### 7.2 An explicit combinatorial bound

If a template has  $k$  still-undetermined data parameters and the arena contains at most  $t$  candidates for each, it has at most  $t^k$  candidate groundings before type and pattern checks. For a finite registry the sum of these bounds is finite. The generated atom set consists only of the instantiated premises and conclusions, together with known and requested atoms.

This bound is intentionally coarse. It shows what must be bounded, not that enumeration is a good implementation. Discrimination trees, head-symbol indices, demand slicing, and memoized instantiation can reduce the actual search. A practical profile also caps the number of admitted instances and arena terms. Exceeding a cap yields an incomplete grounding result, and the completeness claim of Theorem 6.1 is then only for the materialized rules, not for all possible registered instances.

### 7.3 Do not hide synthesis inside matching

A rule such as

$$(\exists y, R(x, y)) \rightarrow P(x)$$

can be a legitimate registered theorem. Its existential premise remains a proposition to prove. A theorem requiring an explicit  $y$  does not authorize the property engine to invent a meaning-bearing witness by an unbounded search disguised as unification. The author or an explicit synthesis request must delegate that construction.

Likewise, a ring normalizer may establish an equality between two fixed expressions. It should return a checked proof as an auxiliary leaf or rule application. It must not change the scalar ring, choose a different branch, or replace exact equality by approximate agreement during matching.

### 7.4 The automatic fragment has an escape hatch, not a truth oracle

Higher-order applications, difficult dependent transports, quantified analytic statements, and unsupported typeclass configurations remain ordinary Lean goals. A selected tactic can attempt



them; a human can supply a proof; a synthesis service can propose a candidate. The finite engine may restart with a new *proved* leaf. It does not infer falsity from failure.

The default explanation should name the consuming operation and the actual missing proposition. It may show several sufficient routes, with the cheapest displayed first, but it must identify the profile under which those alternatives were computed. A display such as “this quotient-transport route needs  $d \mid n$ ” is preferable to “ $d \mid n$  is mathematically necessary” unless that necessity itself has been proved.

## 8 Worked proof: differentiating an identity without losing its region

### 8.1 The exact theorem to preserve

Let  $U \subseteq \mathbb{R}$  be open and let  $W, \varphi : \mathbb{R} \rightarrow \mathbb{R}$ . Suppose

$$\forall x \in U, \quad W'(x) = \varphi(W(x))$$

with the equality supplied as a genuine derivative witness, not merely an equality of totalized derivative values. Let  $G_n, H_n : \mathbb{R} \rightarrow \mathbb{R}$  satisfy, for all  $x \in U$  and all  $n \in \mathbb{N}$ ,

$$G_0(W(x)) = W(x), \tag{7}$$

$$\text{HasDerivAt}(G_n, H_n(W(x)), W(x)), \tag{8}$$

$$G_{n+1}(W(x)) = H_n(W(x))\varphi(W(x)). \tag{9}$$

Then

$$\forall n \in \mathbb{N} \forall x \in U, \quad W^{(n)}(x) = G_n(W(x)). \tag{10}$$

This is the content and locality of the directly inspected *ProveIt* theorem, with  $H_n$  used here for its explicitly supplied derivative family [3].

### 8.2 The human proof and its formal obligations

Induct on  $n$ , keeping the assertion universally quantified over  $U$ . The base case is (7). In the successor step, fix  $x \in U$ . Since  $U$  is open, the induction hypothesis identifies  $W^{(n)}$  and  $G_n \circ W$  on a neighborhood of  $x$ . Equal functions on a neighborhood have the same derivative at  $x$ . The chain rule gives the derivative of  $G_n \circ W$  as  $H_n(W(x))\varphi(W(x))$ , and (9) identifies this with  $G_{n+1}(W(x))$ .

The routine obligations are openness, membership, the neighborhood equality, the two derivative witnesses required by the chain rule, and the recurrence specialization. The strategic decision is to induct on an *on-set identity*. That decision belongs in the source; it should not be reconstructed by an opaque search after an incorrectly specialized induction hypothesis has already lost the information.

### 8.3 Proposed Alder source

```

show forall n, On U (D^n W = G n . W)
proof by induction n maintaining On U
  case zero
    use G_zero
  case successor n IH
    locally at x in U
      differentiate IH using chain_rule
    use G_recurrence

```

Here `OnU` is a controlled binder for a universally quantified on-set assertion. `locally` is a scope-generating proof move. It asks the selected local-differentiation rule to discharge its neighborhood premise from openness and membership. It does not change the meaning of  $D$  globally, and it does not infer differentiability merely from equality of derivative values.

A failed version that keeps only the induction hypothesis at one point should expose

$$\text{requested: } W^{(n)} = G_n \circ W \text{ on a neighborhood of } x,$$

not quietly differentiate the point equality. The counterexample  $f(t) = t$ ,  $g(t) = 0$  at  $t = 0$  demonstrates why that distinction matters.

## 8.4 Ordinary Lean remains an important control

The source’s crucial bridge is already concise:

```

have heq : iteratedDeriv n W =f[nhds x] G n . W :=
  eventuallyEq_of_mem (hs.mem_nhds hx) (fun y hy => ih y hy)
-- derivative transport, followed by the chain rule

```

This listing is an ASCII presentation of the idea, not a claimed Lean transcription. The actual source uses Lean’s eventual-equality notation and named derivative lemmas [3]. A project-owned helper that directly packages the neighborhood bridge may explain most of the authoring improvement. The evaluation must give that helper to the ordinary-Lean control as well.

Alder’s additional hypothesis is that the same use-site interface can request local evidence, display the missing region when it fails, and preserve the invariant through edits without the author repeatedly naming the bridging lemmas. No productivity result is inferred from this attractive example alone.

## 9 Worked arithmetic: exact quotient transport with a satisfiable contract

### 9.1 Separate balance, exact division, and inversion

Let  $n, d, q \in \mathbb{Z}$  satisfy  $d \neq 0$  and  $d \mid n$ , with  $q = n/d$  the exact integer quotient. Then  $dq = n$ . A ring homomorphism  $\iota : \mathbb{Z} \rightarrow R$  preserves that balance:

$$\iota(d)\iota(q) = \iota(n).$$

If  $\iota(d)$  is a unit, one can solve for  $\iota(q)$ . In a field this becomes the familiar quotient formula when  $\iota(d) \neq 0$ . These are separate contracts. Integer divisibility does not by itself make the image of the denominator invertible, especially in a target of positive characteristic.

A proposed exact-quotient interface therefore returns the integer value and a balance proof; a cast-to-field adapter requests the additional target-side unit or nonzero evidence. The ordinary integer division operation remains available with its original semantics when exactness is not known. ALDER does not redefine all divisions as partial operations.

## 9.2 The Frey coefficient guards

Let  $a, b \in \mathbb{Z}$  and  $p \in \mathbb{N}$ , with

$$4 \leq p, \quad p \text{ odd}, \quad a \equiv 3 \pmod{4}, \quad 2 \mid b. \quad (11)$$

Define

$$N_2 = b^p - 1 - a^p, \quad N_4 = -a^p b^p.$$

Then  $4 \mid N_2$  and  $16 \mid N_4$ . For the first claim,  $2 \mid b$  and  $p \geq 4$  imply  $b^p \equiv 0 \pmod{4}$ ; oddness and  $a \equiv -1 \pmod{4}$  give  $a^p \equiv -1 \pmod{4}$ . Hence  $N_2 \equiv 0$ . For the second,  $b = 2k$  gives  $16 \mid b^p$ , and multiplication preserves divisibility.

Only the second and third assumptions in (11) are needed for the first coefficient, not for the second. A per-operation property profile should request the actual premises of the selected theorem rather than attach the entire package to every use. The detailed source proof obtains stronger arithmetic facts from a **FreyPackage**; the syntheses correctly recommend extracting this weaker satisfiable interface for evaluation [5, 1, 2].

The fixture  $(a, b, p) = (3, 2, 5)$  gives

$$N_2 = -212, \quad N_4 = -7776, \quad N_2/4 = -53, \quad N_4/16 = -486.$$

No Fermat equation, primality, or coprimality is used. Thus the test does not depend on an inconsistent full counterexample package from which every proposition would follow.

## 9.3 Alder's requested proof, not an invented arithmetic oracle

```
fix a b : Int
fix p : Nat where 4 <= p, Odd p
assume a == 3 mod 4, Even b

let a2 := exactQuotient (b^p - 1 - a^p) 4
let a4 := exactQuotient (-a^p * b^p) 16

calculate cast a2 = (cast b^p - 1 - cast a^p) / 4
calculate cast a4 = (-cast a^p * cast b^p) / 16
in Rat
```

The registration for `exactQuotient` asks for divisibility and the chosen quotient semantics. The `cast` registration asks for the destination-side guard and uses an actual `cast` theorem. Existing arithmetic tactics can discharge the finite congruence and numeral goals. The required functionality is the composition of these interfaces, not a new general algebra engine.

Removing evenness must not allow the system to pretend the quotient is exact. Changing the target field to characteristic two must not permit inversion of the image of 4. Replacing  $p$  by an even value can break the first coefficient while leaving the second one valid. These mutations should produce different residuals rather than a single package-level failure.

## 9.4 What the tiny companion does and does not test

The supplied quotient example is the propositional support graph in Section 6. It tests parsing, alternative support calculation, proof assembly, and scope restoration. It does *not* implement integer arithmetic, the Frey theorem registry, cast elaboration, or a real **requires** binder. The full arithmetic proof here is a paper proof; the fresh executable experiment is the finite obligation mechanism. The round-three reports separately describe their own compiled Frey helpers.

## 10 Worked hybrid example: a unique formal root and certified approximations

### 10.1 The source construction and the desired interface

The generic ProveIt construction begins with a commutative ring  $R$ , a polynomial jet  $J \in R[Y]$ , and correction weights  $b : \mathbb{N} \rightarrow R$ . Under  $b_0 = 1$ , the coefficient-reduction theorem identifies the parameter-constant part of the constructed germ polynomial with the coefficients of  $J$ . Consequently,

$$J(0) = 0, \quad J'(0) \in R^\times$$

are sufficient for the existing formal-root theorem to supply a unique root with zero constant coefficient [4]. The notation  $J'(0) \in R^\times$  here means its invertibility as a ring element, matching the source's `IsUnit(J.coeff1)`.

ALDER should make the theorem-backed transformation of these guards part of the operation's type. It should not repeatedly wrap  $J$  in nested subtypes, or ask the author to repeat the same coefficient-reduction proof at every use of the resulting root. A local root interface records the defining equation and the branch predicate together. Uniqueness is relative to that predicate.

### 10.2 A concrete instance from the source

The inspected file specializes to

$$J(Y) = Y + 4Y^2, \quad b_0 = 1, \quad b_1 = -\frac{1}{18}, \quad b_j = 0 \ (j \geq 2),$$

and obtains

$$A(Y, Q) = Y + 4Y^2 - \frac{4}{9}Q. \quad (12)$$

Let  $\Delta \in \mathbb{Q}[[Q]]$  be the unique root of  $A(\Delta, Q) = 0$  with  $\Delta(0) = 0$ . This is a formal object, not initially a promise about evaluation at a real argument.

**Proposition 10.1** (Coefficients and branch). *The unique zero-constant root of (12) is*

$$\Delta(Q) = \sum_{n \geq 1} (-1)^{n-1} C_{n-1} \frac{4^{2n-1}}{9^n} Q^n, \quad C_j = \frac{1}{j+1} \binom{2j}{j}. \quad (13)$$

Equivalently, in formal power series over  $\mathbb{Q}$ ,

$$\Delta(Q) = \frac{\sqrt{1 + (64/9)Q} - 1}{8},$$

where the square root has constant coefficient 1.

*Proof.* Write  $\Delta = \sum_{n \geq 1} a_n Q^n$ . Matching coefficients gives

$$a_1 = \frac{4}{9}, \quad a_n = -4 \sum_{i=1}^{n-1} a_i a_{n-i} \quad (n \geq 2).$$

This determines exactly one sequence of coefficients, and coefficientwise substitution proves that the resulting formal series is a root. Put  $a_n = (-4)^{n-1} (4/9)^n c_{n-1}$ . The recurrence becomes  $c_0 = 1$  and  $c_j = \sum_{i=0}^{j-1} c_i c_{j-1-i}$ . Its generating series  $C(t)$  satisfies  $C = 1 + tC^2$ , whence the branch with constant coefficient 1 is  $(1 - \sqrt{1 - 4t})/(2t)$ . Expanding the formal binomial series gives  $c_j = \binom{2j}{j}/(j+1)$ , proving (13). The displayed radical formula follows by the quadratic formula with the constant-one square-root branch, or by direct substitution and uniqueness.  $\square$

The first terms are

$$\Delta(Q) = \frac{4}{9}Q - \frac{64}{81}Q^2 + \frac{2048}{729}Q^3 - \frac{81920}{6561}Q^4 + \dots$$

This paper derivation is independently checked to 33 coefficients by the supplied exact-rational companion. That finite check is not the proof of the all-orders formula; the recurrence argument above is.

### 10.3 A residual certificate needs a branch-sensitive bridge

A CAS may propose a polynomial  $S_N(Q)$  and show

$$A(S_N(Q), Q) \equiv 0 \pmod{Q^N}.$$

Why does this certify the intended root to order  $N$ ? A zero residual alone does not specify a branch. The second exact root of (12) is  $-1/4 - \Delta$ , whose constant coefficient is not zero. A certificate consumer must require  $S_N(0) = 0$  and use a local uniqueness argument.

**Theorem 10.2** (Formal residual-to-root certificate). *Let  $R$  be a commutative ring and let  $A(Y) = \sum_{k=0}^d a_k Y^k$  with  $a_k \in R[[Q]]$ . Suppose the constant coefficient of  $a_1$  is a unit of  $R$ . If  $S, T \in R[[Q]]$  both have constant coefficient zero and  $A(T) = 0$ , then for every  $N \in \mathbb{N}$ ,*

$$A(S) \in (Q^N) \iff S - T \in (Q^N).$$

*In particular, a checked residual below order  $N$  certifies equality to the chosen zero-constant root below order  $N$ .*

*Proof.* The difference-of-powers identity gives

$$\begin{aligned} A(S) - A(T) &= (S - T)U, \\ U &= \sum_{k=1}^d a_k \sum_{j=0}^{k-1} S^{k-1-j} T^j. \end{aligned}$$

Since  $S(0) = T(0) = 0$ , the constant coefficient of  $U$  is exactly that of  $a_1$ . Hence  $U$  is a unit in  $R[[Q]]$ . To see the unit criterion directly, choose the inverse of its constant coefficient and determine subsequent coefficients of the inverse recursively from the equation  $UV = 1$ ; only multiplication by that already invertible constant is required.

Multiplication by a unit preserves and reflects membership in the ideal  $(Q^N)$ . Using  $A(T) = 0$  in the factorization proves the equivalence. No field structure on  $R$  and no cancellation of an arbitrary nonzero element is used.  $\square$

For (12), the factor is  $U = 1 + 4(S + T)$  and its constant coefficient is 1. The theorem is thus particularly simple. It turns a finite coefficient certificate into a finite-order statement about the exact distinguished root, while keeping the original branch condition visible.

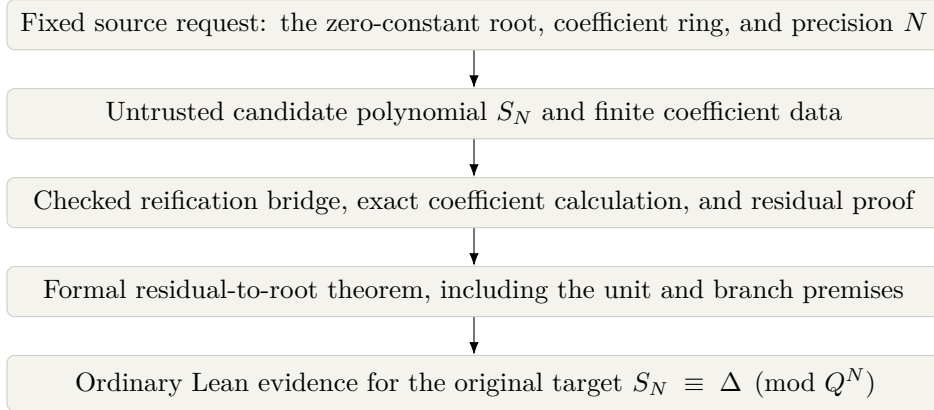
### 10.4 Why nonzero is not enough

Over  $\mathbb{Z}$ , the polynomial  $2Y - Q$  has nonzero, regular linear coefficient 2, but no formal root with integer coefficients: its coefficient of  $Q$  would require  $2a_1 = 1$ . Over  $\mathbb{Z}/4\mathbb{Z}$ , the polynomial  $2Y$  has both 0 and  $2Q$  as zero-constant roots, although its linear coefficient is nonzero. The missing premise is invertibility, not merely nonzeroness.

A type system that treats “nonzero” as a universal license to invert will therefore mis-handle precisely the sort of formal algebra that this example exercises. `ALDER` keeps `Nonzero`, `IsRegular`, `IsUnit`, and an actual computational unit witness as different interfaces. In a field some implications are proved by the library; they are not built into an unqualified rule for every ring.

## 10.5 The complete certificate path

The intended end-to-end path is:



A checker theorem about an encoded polynomial is not enough without the bridge between that encoding and the original source expression. Atom identities, variable order, coefficient ring, and arity are checked before combining coefficient arrays. Matching display names or relying on a digest as a proof of equality is insufficient.

Existing proof-producing algebra tactics are legitimate providers for the calculation. A new sparse-polynomial checker is not a prerequisite for every ALDER feature. If a new checker is introduced, its denotation and soundness theorems must be proved, and its execution path must fit the project's axiom policy. A native computation is not accepted merely because the abstract algorithm has a correctness theorem; the claimed result of that particular execution must be justified too.

## 10.6 Analytic information is a separate upgrade, not a badge on the jet

For this particular root, an analytic upgrade is available by a separate argument. Let  $a_n$  be the coefficients in (13). For  $n \geq 1$ ,

$$\left| \frac{a_{n+1}}{a_n} \right| = \frac{16}{9} \frac{4n-2}{n+1} \rightarrow \frac{64}{9}.$$

Thus the power series has radius of convergence  $9/64$ . For  $|q| < 9/64$  and  $N \geq 1$ , the same ratio bound gives

$$\left| \Delta(q) - \sum_{n=1}^{N-1} a_n q^n \right| \leq \frac{|a_N q^N|}{1 - (64/9)|q|}. \quad (14)$$

Indeed, each subsequent term has absolute size at most  $(64|q|/9)$  times its predecessor; summing the geometric majorant proves the bound. Substitution is legitimate by convergence, so this analytic series satisfies the equation and selects the branch near zero.

The formal statement  $S_N \equiv \Delta \pmod{Q^N}$  does not imply (14) without this additional convergence and tail argument. In a proposed ALDER interface, asking for a real error bound would generate those analytic obligations, not silently promote the formal residual certificate. This is an example of a useful stronger property type whose evidence is explicit even when its routine derivation is automated.

## 11 Behavioral synthesis over realizable observation frames

### 11.1 The limitation of comparing raw affine coefficients

Suppose a candidate's output length is an affine function of observed input lengths. Equality of two coefficient vectors is a sufficient test for equality on all length vectors. It need not be necessary on the vectors the source type can actually realize.

For a single `ListEmpty`, only length 0 is possible. The affine expressions 0 and  $n$  therefore describe the same output-length law on that domain. For the pair of observations  $(\text{length}(xs), \text{length}(\text{reverse}(xs)))$ , only the diagonal  $(n, n)$  is possible. The expressions  $n_1$  and  $n_2$  agree there despite different coefficients. A Cartesian product of marginal possibilities would forget exactly this correlation.

The round-three reports separated positive soundness, decision completeness, negative realization, and observation coverage. The following theorem gives a concrete interface that supplies all the needed directions for a useful equality fragment without requiring the full observation image to be enumerated [1, 2].

### 11.2 A realizable affine frame

**Definition 11.1** (Affine observation frame). Let  $X$  be a source input type,  $D : X \rightarrow \text{Prop}$  an admissibility predicate, and  $\text{obs} : X \rightarrow \mathbb{Z}^d$  an observation map. A realizable affine frame consists of admissible inputs  $x_0, \dots, x_m$  such that every admissible  $x$  satisfies

$$\text{obs}(x) = b + \sum_{j=1}^m k_j v_j \quad \text{for some } k_j \in \mathbb{Z}, \quad b = \text{obs}(x_0), \quad v_j = \text{obs}(x_j) - b. \quad (15)$$

The data includes proofs of  $D(x_i)$  and of this coverage statement. It need not assert that every integer combination in (15) is itself realizable.

The last sentence matters. A finite interval of possible lengths can have the same affine frame as all natural lengths. A diagonal observation family can have a one-direction frame although its ambient space is two-dimensional. The coverage used for positive reasoning may be an overapproximation, provided that the actual test points have concrete admissible realizations.

**Theorem 11.2** (Finite decision by realizable affine observations). *Let  $G$  be an abelian group, and let  $L, R : \mathbb{Z}^d \rightarrow G$  be affine maps: a constant plus an additive homomorphism. If  $x_0, \dots, x_m$  is a realizable affine frame, then*

$$\forall x, D(x) \rightarrow L(\text{obs}(x)) = R(\text{obs}(x)) \iff \forall i \in \{0, \dots, m\}, L(\text{obs}(x_i)) = R(\text{obs}(x_i)).$$

*Any failing frame point is a concrete counterexample to the universal equality. If equality in  $G$  is decidable, the finite comparison is a decision procedure; without that additional assumption, the result is a finite logical reduction rather than an executable decision procedure.*

*Proof.* The forward implication follows by specialization to each admissible  $x_i$ . For the reverse implication put  $H = L - R$ , so  $H(z) = c + \ell(z)$  with  $\ell$  additive. Equality at  $x_0$  gives  $H(b) = 0$ . Equality at  $x_j$  gives

$$0 = H(b + v_j) = H(b) + \ell(v_j),$$

so  $\ell(v_j) = 0$  for every direction. For any admissible  $x$ , use (15) and additivity, including integer multiples, to obtain

$$H(\text{obs}(x)) = H(b) + \sum_{j=1}^m k_j \ell(v_j) = 0.$$

A failing test is already attached to an admissible  $x_i$ ; specialization would contradict the claimed universal law. No surjectivity onto  $\mathbb{Z}^d$  is required.  $\square$



### 11.3 Source semantics must commute with the observation model

Theorem 11.2 concerns affine maps. A candidate program is connected to it by a separate denotation theorem. For instance, for an exact candidate  $c$  and source specification  $s$  one needs

$$\begin{aligned}\forall x, D(x) \rightarrow \text{length}(c(x)) &= L_c(\text{obs}(x)), \\ \forall x, D(x) \rightarrow \text{length}(s(x)) &= L_s(\text{obs}(x)),\end{aligned}$$

with lengths embedded into  $\mathbb{Z}$  when necessary. Only then does the finite frame comparison prove the source-level equality of lengths. It still says nothing about element order or permutation unless those have their own sound models and bridges.

An arbitrary black-box implementation that agrees on the frame can behave differently elsewhere. The affine model is justified by a source grammar or a theorem about that exact implementation. Finite tests alone do not supply it.

### 11.4 The list grammar and its normalization theorem

Consider a grammar with list variables, the empty list, cons by a fixed element, append, reverse, and map by a fixed total element function. Let  $n_i$  be the input lengths. Associate an affine expression with each syntax node:

$$\begin{aligned}\lambda([]) &= 0, & \lambda(x_i) &= n_i, \\ \lambda(a :: e) &= 1 + \lambda(e), & \lambda(e_1 ++ e_2) &= \lambda(e_1) + \lambda(e_2), \\ \lambda(\text{reverse}(e)) &= \lambda(e), & \lambda(\text{map}(f, e)) &= \lambda(e).\end{aligned}$$

Combining constants and coefficients computes an affine normal form.

**Proposition 11.3** (Length-model soundness). *For every well-typed expression  $e$  in this grammar and every environment of lists,*

$$\text{length}(\text{eval}(e, \text{env})) = \lambda(e)(\text{length}(\text{env}_1), \dots, \text{length}(\text{env}_d)).$$

*Proof.* Induct on the syntax of  $e$ . Empty lists and variables are immediate. Cons adds one to length; append adds the two lengths; reverse and map preserve length. Substitution of the induction hypotheses gives the stated affine equations. Coefficient collection preserves evaluation by associativity and distributivity of integer addition and multiplication.  $\square$

The grammar must itself be type-correct. A cons node over **Empty** requires an actual element of **Empty**; it cannot appear in a closed admissible program merely because an untyped length model permits “add one.” The Python interpreter uses lists of natural numbers and does not pretend to enforce Lean’s element-type constraints for arbitrary external programs.

### 11.5 Three frames, three different correct answers

For  $d$  independent **ListNat** inputs, take the all-empty environment and the  $d$  environments with a singleton zero list in one coordinate. Their observations are  $0, e_1, \dots, e_d$ . They cover all natural length vectors by nonnegative integer combinations and hence by (15). Equality of affine laws is equivalent to equality on these  $d + 1$  realizable probes.

For independent **ListEmpty** inputs, take only the all-empty environment. The direction family is empty. Every actual observation is zero, and only equality of constant terms matters. Comparing all raw coefficients would reject some true laws.

For the correlated observations of a list and its reverse, use the empty list and a singleton zero list. The frame observations are  $(0, 0)$  and  $(1, 1)$ . An affine difference  $c + an_1 + bn_2$  vanishes

on all admissible inputs exactly when  $c = 0$  and  $a + b = 0$ , not necessarily when  $a = b = 0$ . A model-relative witness such as  $(1, 0)$  is not realizable and must not refute the source claim.

When  $D$  has no admissible input, the universal claim is vacuous. No realizable base point  $x_0$  exists. A separate proof of emptiness handles that case. An empty Python list of test points does not by itself certify that the source domain is empty.

## 11.6 Why the coefficient domain matters

The theorem uses integer affine combinations and an abelian group codomain. A rational linear-span test is a different theorem. Clearing denominators and then canceling an integer multiple can fail in a group with torsion. For example,  $2g = 0$  need not imply  $g = 0$ .

Similarly, equality is essential to the affine-span argument. Agreement of inequalities at a collection of points does not extend to arbitrary signed affine combinations. Bounds can instead use a convex-hull coverage theorem with nonnegative weights, or a separately proved monotonicity argument. ALDER records the exact transport contract rather than treating every numeric observation as the same kind of evidence.

## 11.7 How this fits Leant without replacing its existing design

The Length source contract already carries exact spine identities, argument roles, provider transfers, and a joint pair-contract form [9]. An ALDER adapter can augment that interface with:

1. A theorem linking the exact candidate's source semantics to the selected abstract transfer, including joint outputs.
2. A proof-backed frame or another explicitly stated coverage/realization interface for the admissible input domain.
3. A retained derivation connecting the successful finite check to the original frozen behavioral target.

These are additional authorities, not inferred meanings of existing metadata. Ranking can continue to use inexpensive untrusted models. Acceptance requires the bridge. A negative outcome can be labeled `model-relative counterexample` until the witness is realized at the source type; it becomes a `checked candidate refutation` only after the exact negated source claim is proved.

The finite-frame theorem offers a sharper integration target than “use SMT to understand behavior.” It identifies a grammar, a domain theorem, a finite computation, and a precise source-level result. It also supplies a meaningful regression suite: independent, empty, and correlated input domains must yield different notions of equality.

# 12 Relations, scopes, and mathematical proof moves

## 12.1 Transport belongs to the consumer

An evidence entry is not just “ $f$  equals  $g$ .” It contains the actual proposition: equality of functions, equality on a set, eventual equality at a filter, almost-everywhere equality for a measure, or agreement below a power-series order. A registration states how a specified consumer respects that relation.

For example, neighborhood equality permits derivative transport at the neighborhood's point. Almost-everywhere equality permits the corresponding integral-congruence theorem for the fixed measure. A change of measure requires its own transport theorem. A coefficient truncation supports calculations in the relevant quotient ring. None of these relations is globally substituted for equality of all observations.

The exact premises come from the actual theorem signature. Some totalized operations admit

congruence without existence or integrability assumptions that a more informative analytic result would require. ALDER should not impose generic guards that the selected theorem does not need; conversely, it must not omit the guards of a theorem asserting an actual derivative or a convergent sum.

## 12.2 A useful nontrivial upgrade from almost everywhere

There are legitimate routes from almost-everywhere to pointwise equality. A paper-level registration example is the following.

**Proposition 12.1** (Continuity and full support). *Let  $X$  be a topological space equipped with a measure  $\mu$  such that every nonempty open set has positive measure. If  $f, g : X \rightarrow \mathbb{R}$  are continuous and  $f = g$  almost everywhere with respect to  $\mu$ , then  $f = g$  everywhere.*

*Proof.* If  $f(x_0) \neq g(x_0)$ , continuity of  $f - g$  implies that the set on which  $f - g$  is nonzero is an open neighborhood of  $x_0$ . It therefore has positive measure. Almost-everywhere equality says exactly that this disagreement set has measure zero, a contradiction.  $\square$

The full-support premise is necessary for this route: a measure concentrated at one point need not distinguish continuous functions away from that point. A registered upgrade using Proposition 12.1 would permit subsequent point evaluation, but only after producing the additional evidence. This demonstrates why the property interface should be extensible by theorems, not hard-coded as a list of permanently incomparable badges.

## 12.3 “Without loss of generality” has a typed interface

Many human proofs are shortened by symmetry. The abbreviation is justified when the language records both a normalization route and invariance of the target.

Let a group  $G$  act on  $X$ , let  $P : X \rightarrow \mathbf{Prop}$  be the target, and let  $N$  be a normal-form predicate. Suppose

$$\forall x, \exists g : G, N(g \cdot x), \quad \forall g x, P(g \cdot x) \leftrightarrow P(x).$$

Then proving  $\forall x, N(x) \rightarrow P(x)$  suffices to prove  $\forall x, P(x)$ . Choose the normalization witness for the fixed  $x$ , apply the normal-form theorem, and transport back by invariance. The witness remains scoped to this proof. No executable choice of normalizer follows merely from its proposition-valued existence.

A proposed command `wlogNbysymmetryaction` elaborates through precisely such a theorem. It does not guess that an expression looks symmetric. For a two-variable symmetric statement in a linear order, the chosen normal form might be  $a \leq b$ , with the swap action and an explicit proof that swapping preserves the claim.

## 12.4 Precision is an index, not a descriptive adjective

Let  $S \equiv_N T$  mean agreement below order  $N$ . Differentiation can use

$$S \equiv_{N+1} T \implies S' \equiv_N T'.$$

It generally cannot preserve  $N$ :  $Q^N$  and 0 agree below  $N$ , whereas their derivatives can differ at degree  $N - 1$ . A request for an output jet of order  $N$  can therefore propagate a demand for an input jet of order  $N + 1$ . The requested output precision remains fixed; obtaining more input data is a delegated computation, not a change of statement.

This is an example of a requirement transformer already identified in earlier proposals. ALDER does not claim the idea as new. Its residual discipline supplies a uniform place to display the increased precision demand and a fixed boundary at which a certified producer may discharge it.

## 13 Definition packages and universal properties

### 13.1 A definition should export an evidence interface

A mathematical definition package contains a carrier-level construction, its input structures, and separately named theorems about its result. It may also contain algorithms with proved correspondence to that construction. An author can request the part of the interface needed by the next proof step.

For a finite tensor-family construction, the data and guarantees should be separated conceptually:

```
construct W representing (T |-> forall i, AlgHom A (H i) T)
  using finite_tensor_family

request FiniteModule A W
request FreeModule A W
request Natural representing_equivalence
```

The representability request fixes the functor and the category of algebras. Finiteness and freeness are separate properties of the same constructed object. The naturality assertion concerns the entire family of equivalences and its behavior on algebra maps.

The inspected Lean source includes finite and free hypotheses for every factor when it requests those module properties [6]. For a singleton family, the representing object is isomorphic to the sole factor. Taking  $A = \mathbb{Q}$  and  $H = \mathbb{Q}[X]$  shows that finite indexing alone cannot guarantee a finite-dimensional representing algebra. The property layer should surface the missing factor premises; it must not use a smaller index set as a proxy for a module bound.

### 13.2 Bundling is a boundary decision

Unbundled objects with local property proofs are often convenient while developing a proof. Bundled morphisms, subobjects, equivalences, and structures are often convenient at interfaces. ALDER should bridge these representations using explicit constructors and projection theorems.

For instance, a function together with a proved homomorphism law can be packaged as a homomorphism. A family together with a proved naturality law can be packaged as a natural transformation. In each case the construction must preserve the exact underlying function or family. It is not acceptable to synthesize a different morphism that merely satisfies a similar law and then pretend the original object has been refined.

Different constructions satisfying a universal property can be related by a canonical isomorphism *after proving the relevant uniqueness theorem*. That is not definitional identity of their carriers. Dependent expressions on those carriers still require the isomorphism-specific transport appropriate to their consumer.

### 13.3 Property learning is monotone; witness selection need not be

Adding a new proof of  $P(t)$  enriches the local evidence about the same  $t$ . Selecting a normalized representative, choosing a root, extending a partial function, or replacing a presentation by an isomorphic one introduces data. These operations may change subsequent computations and dependent types.

The source distinguishes `know` from `construct` for exactly this reason. A merge can usually combine new independent property proofs. Competing choices of a witness or structure can conflict, even when both choices are mathematically valid. A proof of uniqueness may later identify them propositionally, but a merge tool should not invent that proof or infer definitional equality from it.

## 14 Implementation architecture and incremental correctness

### 14.1 An explicit pipeline

The proposed Lean integration has six stages. First, parse the mathematical moves and elaborate their meaning-bearing terms. Second, record the fixed target, structure arguments, context, and allowed witness holes. Third, instantiate validated theorem registrations into a bounded graph. Fourth, solve proof obligations using the finite engine and selected external proof services. Fifth, reconstruct and check a Lean term for the exact target. Sixth, attach a source-linked receipt to the completed assertion or display its remaining conditional routes.

The output of one stage is typed input to the next. A solver cannot enlarge its authority by returning a different result kind. A candidate producer returns proposed data and evidence, not a declaration that it has changed the theorem. The final checker checks the exact elaborated target, not a string similarity score or a more convenient reformulation chosen by the producer.

### 14.2 Evidence-index entry and identity

A production evidence entry needs at least the following logical content:

( $\Gamma$ -snapshot, proposition, proof, source support, structure footprint, origin).

The proposition is an elaborated Lean expression, not a printed property name. Its arguments include all relevant types, universes, operations, topologies, measures, and dependent indices. The proof must be checked in the current context or imported through a verified context embedding.

Hashes are useful for lookup and invalidation. They are not proofs that two terms, contexts, or structures are equal. Exact expression comparison and kernel-checked transport remain the authority. Proof irrelevance can identify proofs of one proposition; it cannot identify two multiplication operations on the same carrier.

### 14.3 A conservative answer to incremental elaboration

The first implementation should rebuild the local evidence index for each elaborated declaration and discard it when the declaration is invalidated. It can retain global theorem registrations as an environment extension, but local facts must not survive by recycled local-variable identifiers.

A more aggressive cache may abstract a proof over its dependency telescope, retain that closed representation with environment dependencies, and instantiate it into a new context only after checking the substitution and resulting type. This is a typed context embedding problem. Generative epochs, as used in Leant's post-verification boundary, are good protection against accidental cross-batch association, but they do not by themselves prove that a dependent Lean context has been transported correctly [8].

The receipt is bound to a particular target, source edit, environment, and proof. After a dependency or toolchain change, rechecking issues a new receipt. A persistent green badge is not evidence that an old native result or stale local proof is valid in the new environment.

### 14.4 What happens when simplification rewrites an anchor

There are three cases. If the old and new elaborated expressions are definitionally equal in the current context, the proof can be reused through ordinary type checking. If simplification supplies a propositional equality or iff, create a transport edge and keep the old entry rather than silently changing its anchor. If neither applies, the new expression requires new evidence.

For dependent indices, the transport itself can change the type of the object. An element of  $\text{Fin}(n + 1)$  is not an element of  $\text{Fin}(m)$  merely because a printer displays similar arithmetic. A

proof of  $n + 1 = m$  allows a dependent cast, and the consumer must see the cast or a proved equivalent interface. Simplifier normal forms are not globally canonical object identities.

The initial implementation can be conservative: keep proof-carrying rewrite edges only within the current declaration and reindex after a major goal transformation. The cost can then be measured before designing a persistent congruence structure for every kind of property.

## 14.5 Proof sharing and the cost of elaboration

Repeatedly inlining the same transport or guard can create large proof trees even when the source is short. Store proof DAGs and emit shared local bindings. Export useful repeated derivations as helper theorems when appropriate. Record at least the number of nodes, retained support sets, rule instantiations, generated term size, and time spent in final checking.

This does not establish that sharing is always a net win. A large index can cost more than calling an existing tactic. An aggressively opaque helper can change simplification behavior. The ordinary-Lean baseline must receive the same helper theorems and services, so the measurement isolates the interface rather than rewarding ALDER for owning a better library.

## 14.6 Structure contexts do not erase instance search problems

A lexical structure context fixes the relevant dictionaries for a block and can make their choices inspectable. It may reduce repeated setup and accidental changes. It cannot be assumed to remove all generated instance-disabling preambles in the Fermat corpus: those preambles may reflect elaboration search interactions not solved by merely naming a dictionary.

The proposed experiment is to instrument one real file, preserve the selected structures explicitly, and compare the remaining setup and search behavior. This article does not report such a run. It also does not register every learned proposition as a global typeclass instance, which would risk recreating the very search pollution the layer is meant to avoid.

# 15 Should Lean’s kernel type system change?

## 15.1 The first extension can be real without being trusted

A source-level `requires` binder is more than a comment: the typechecker generates and discharges an obligation at each application, and the resulting declaration has an explicit proof argument in Lean. A behavioral arrow is more than an annotation: crossing the interface requires a function together with a theorem about that exact function. A residual type is more than an error message: it denotes a conditional term whose telescope can be checked.

These are meaningful type-system extensions at the authoring level. Their translation can use ordinary Lean  $\Pi$ -types, propositions, subtypes, structures, and equality elimination. This is the recommended implementation, because the remaining risks are predominantly in elaboration, dependency tracking, and usability, not in logical expressiveness.

## 15.2 An explicit-evidence primitive refinement former

A candidate kernel experiment would add a primitive type  $\text{Ref}(A, P)$  with rules equivalent to

$$\frac{a : A \quad h : P(a)}{\langle a, h \rangle : \text{Ref}(A, P)}, \quad \frac{r : \text{Ref}(A, P)}{\text{val}(r) : A}, \quad \text{property}(r) : P(\text{val}(r)).$$

Its essential computation rules would reduce projections on constructors. Subsumption would still require evidence:

$$\text{weaken}(r, k) : \text{Ref}(A, Q), \quad k : \forall a, P(a) \rightarrow Q(a).$$



The expected ordinary-Lean translation is a subtype. An explicit introduction without  $h$  would merely move the missing proof somewhere else, or change the logic.

Crucially,  $\text{Ref}(A, P)$  must not be definitionally identified with  $A$ . With  $P$  constantly false and  $A$  inhabited, such an identification together with a property projection would manufacture a proof of false. Erasing the proof field from compiled runtime representation is not the same as identifying the types in the kernel.

### 15.3 What could a primitive improve?

A primitive might make certain elaboration annotations cheaper to retain, make a projection representation more uniform, or enable a carefully restricted normalization of coercion composition. Those are engineering hypotheses. Ordinary Lean already erases proposition proofs from executable code and identifies proofs of the same proposition through proof irrelevance [10]. Many same-carrier refinement paths therefore have little or no computational distinction to remove.

A genuine cost case would have to exhibit repeated dependent transports or coercion-normalization work that optimized ordinary encodings cannot remove. It would then compare the primitive against those optimized encodings, including shared proofs and better theorem interfaces. Without that experiment, “kernel refinement types will be faster” is not an evidence-backed conclusion.

### 15.4 Definitional functoriality is a separate, nontrivial experiment

Some verbosity arises from maps and transports that compose propositionally but do not normalize to the same term. The research on definitional functoriality for dependent subtypes studies adding selected functor laws to definitional equality and justifying structural coercive subtyping; its mechanized result concerns a specifically designed dependent type theory with list functor laws, not a drop-in proof that arbitrary extensions of Lean are safe [17].

For ALDER, a plausible research prototype would choose one container family and explicit coercion maps, formulate the desired computation rules, and prove that translation preserves typing and the new conversions. Merely observing that a law is propositionally provable in Lean is insufficient: using it as conversion under dependent types can require inserted transports. One must analyze coherence, substitution, reduction, and the behavior of the actual checker.

Lean4Lean provides relevant infrastructure and results for studying Lean’s type theory and checker. The current cited paper reports correctness results for parts of the kernel; this is not a blanket completed verification of every future kernel change [18]. A refinement or coercion experiment would need its own metatheoretic and implementation argument.

### 15.5 Semantic subtyping is not the default conversion relation

A judgment  $\text{Ref}(A, P) \leq \text{Ref}(A, Q)$  may be justified by a theorem  $\forall x, P(x) \rightarrow Q(x)$ . The elaborator can search for that theorem and insert a coercion. Installing unrestricted theorem search inside definitional conversion is a different design: ordinary type checking would now inherit the behavior of an open-ended proof search procedure.

A bounded search timeout means unknown, not false. Keeping that search outside conversion allows the kernel to check the explicit successful result using its existing rules. The proposal does not rely on a simplistic claim about all aspects of Lean conversion being decidable; its point is to avoid adding arbitrary mathematical entailment as an implicit conversion obligation.

### 15.6 Computational witnesses must not disappear into propositions

An `IsUnit` proof is a proposition about  $a$ . An actual unit value also carries an inverse, which can be computational data. A classical mathematical construction may recover a witness



noncomputably. An executable operation needs an available constructive witness or a verified algorithm that computes it.

Likewise, existence and uniqueness of a root do not automatically give an executable infinite-series representation. The recurrence in Proposition 10.1 supplies a finite-coefficient algorithm for this example; that algorithm is additional content. ALDER keeps the distinction between a mathematical object with certified properties and a program with certified behavior.

## 15.7 Recommended decision

Implement property-implicit binders, behavioral interfaces, and residual inference above the existing kernel. Treat a primitive refinement former as a measured representation experiment, and definitional functoriality as a separate research project with explicit metatheory. Do not make the authoring-language prototype depend on either. This gives the user richer mathematical types immediately while preserving an ordinary Lean proof artifact as the final authority.

# 16 The executed Alder-0 artifact

## 16.1 The actual grammar

The shipped frontend has the following line-oriented grammar. Identifiers are opaque proposition names, not Lean terms. Comments begin with #.

```
atoms P Q R
rule step : P & Q -> R
assume P
repair Q
explain R
begin local
  assume Q
  show R
end
```

Atom and rule declarations precede proof work. **assume** introduces an explicit hypothesis; **repair** sets the allowed missing-premise vocabulary; **explain** computes alternatives without extending the known facts; **show** requires the empty support. Lexical blocks restore the parent's facts and repair vocabulary when they end. They are scope tests, not an implementation of case elimination.

Every **show** is checked, including an assertion unused by the rest of the document. A module with a failed **show** produces no new output files through the command-line path. Conditional explanations generate explicitly conditional theorem text, never an unconditional proof of their targets.

## 16.2 The independent derivation checker

Proof nodes have three constructors: known assumption, permitted missing premise, and registered rule application. The checker verifies exact active assumption identities, target identity, rule identity, premise order, arity, and recursively computed residual support. It detects cycles and does not trust support annotations computed by the inference engine.

The checker is written separately from the saturation loop. This is useful defense against bookkeeping mistakes, not formal verification of the checker itself. A Python error shared by both implementations remains possible. The finite tests compare the inference result against a third, simple Boolean forward-closure oracle.

### 16.3 Generated Lean and its explicit boundary

A successful proof or residual is rendered as a theorem with opaque proposition parameters and explicit rule hypotheses. A typical output has this shape:

```
theorem sample (P Q R : Prop)
  (r : P -> Q -> R) (hP : P) (hQ : Q) : R := by
  have p0 : R := r hP hQ
  exact p0
```

The real generator shares proof DAG nodes through local **have** declarations and assigns internal identifiers rather than injecting source identifiers into Lean syntax. The files contain no **sorry** placeholders and do not declare the registry’s mathematical laws as axioms. The laws are theorem parameters. Thus the generated terms express the verified *propositional implication structure*; they do not establish that an arbitrary rule name denotes a true fact about integers, derivatives, or roots.

No Lean executable was available in the inspected environment. These generated files were not kernel-checked here. The package gives commands for that additional check, and the article never attributes fresh Lean acceptance to them. The richer ALDER examples elsewhere in the article are a proposed surface, not accepted input to this small parser.

### 16.4 Executed results and their units

The test run used Python 3.13.5 and the standard library. The complete result, including runtime and source hashes, is retained in `evidence/test_results.json`.

| Executed family                                                      | Count  |
|----------------------------------------------------------------------|--------|
| Directed unary rule graphs on three atoms, excluding self-edges      | 64     |
| All known-seed and repair-vocabulary configurations for those graphs | 4,096  |
| Independent support-subset closure probes for that family            | 13,824 |
| Returned derivations independently checked in that family            | 11,904 |
| Deterministically generated five-atom conjunctive rule graphs        | 200    |
| Support-subset oracle probes for the conjunctive family              | 1,617  |
| Returned derivations independently checked in that family            | 934    |
| Expected parser/checker/budget/cycle rejections                      | 16     |
| Accepted authored proof or explanation requests in two examples      | 4      |
| Generated list-grammar expressions                                   | 300    |
| Concrete interpreter versus affine-model comparisons                 | 19,200 |
| Affine-form pair/frame comparisons across three domain kinds         | 46,875 |
| Observation arity rejection                                          | 1      |
| Formal-root coefficients checked against the closed formula          | 33     |
| Perturbed root-jet residual checks                                   | 48     |
| Wrong-branch and nonunit arithmetic controls                         | 2      |

All these tests passed. The rows have different units and should not be summed into a fictitious count of formal theorems. “Exhaustive” applies only to the explicitly enumerated three-atom unary-graph family, not to all Horn theories. The list and affine arithmetic checks are finite executions, not kernel proofs of Propositions 11.3 or Theorem 11.2.

The affine frame tests compare all pairs from 125 affine forms against independent, empty, and diagonal observation domains. The root tests confirm the coefficient formula through degree 32, check zero residual at that truncation, and alter coefficients to ensure the residual detects the change at the expected order. A second exact branch deliberately passes the residual test and

fails the zero-constant branch condition. That is the desired outcome, not a broken checker.

## 16.5 What has advanced beyond a paper policy

There is now authored input with scope, a parser, an explicit finite inference computation, proof DAG construction, a separate validation pass, and generated conditional Lean source. The earlier reports' objection that a property engine was mostly a policy description is addressed for this narrow propositional part.

What has not advanced is equally specific: no mathematical theorem registry is linked to Lean, no actual Lean expression is reified by this frontend, no dependent context is cached across Lean edits, and no user study measures authoring benefit. The unit tests do not exercise imports changing typeclass choices, full `simp` transports, actual universes, or kernel trust-policy migration. Those remain integration tests, not silently counted among the passing Python cases.

## 17 Evaluation that can distinguish a language from better automation

### 17.1 The first production slice

The first Lean implementation should expose one property-implicit binder and one method application around exact quotient transport. Use the satisfiable arithmetic contract of Section 9, a small validated rule registry, and an obligation view. An ordinary Lean version must use the same extracted helper theorems. The next slice should connect the list grammar and a proved affine observation frame to Leant's exact candidate boundary.

Only after those two paths work from authored source to checked original-target evidence should the mathematical surface expand. The autonomous-derivative and formal-root examples in this article are already development material; they are not held-out transfer tasks. A genuinely held-out local-analysis task should be selected by an independent evaluator after the rule registry and interface have been fixed.

### 17.2 Matched controls

The comparison should distinguish the following layers rather than assign every improvement to syntax:

| Arm   | Available capabilities                                                                                         |
|-------|----------------------------------------------------------------------------------------------------------------|
| $L_0$ | Current ordinary Lean with its existing automation.                                                            |
| $L_1$ | The same new mathematical packages and registered helper theorems as Alder, used through ordinary Lean.        |
| $L_2$ | $L_1$ plus the same synthesis, arithmetic, certificate, and search services.                                   |
| $L_3$ | $L_2$ plus property-implicit application and the typed residual interface.                                     |
| $L_4$ | $L_3$ plus the richer mathematical proof-move surface.                                                         |
| $L_R$ | An appropriate ordinary-Lean arm with the same compact read-only renderer, but without the new input language. |

This follows the controlled-comparison logic of the round-three syntheses [1, 2]. Current `fun_prop` already decomposes functions, uses registered function-property theorems, and discharges local hypotheses. Its documentation also identifies transition rules as a source of search growth and provides controls for their depth. It is therefore both a baseline and a source of implementation lessons, not a straw man [11].

### 17.3 What to measure

Record author interventions, time to a correct accepted proof, failure-diagnosis time, edit-repair work, and the reader's recovery of the exact mathematical statement. Record computational quantities separately: admitted rule instances, rule firings, antichain sizes, proof DAG nodes, term size, final checking time, and rebuild cost.

Line count can describe a source artifact but cannot establish usability. A one-line opaque invocation may be harder to understand and maintain than six explicit lines. Conversely, asking a reader to expand every automatically solved guard defeats the intended abstraction. The reading test should ask specifically about hidden-risk distinctions: domain, measure, branch, precision, admissible inputs, uniqueness, and completeness of a solution set.

### 17.4 Paired mutations, with meaningful positive fixtures

Each production acceptance test needs a satisfiable positive fixture, a single specified mutation, and an expected kind of failure. A representative matrix is:

| Positive fixture                | Mutation and required response                                                                      | Status here                        |
|---------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------|
| Open-region derivative identity | Retain only a point identity; local differentiation stays unresolved.                               | Paper/source example               |
| Frey arithmetic at (3, 2, 5)    | Remove a divisibility premise; exact-quotient route exposes the guard.                              | Paper/source example               |
| Zero-constant formal root       | Use the other exact branch; residual success must not certify the requested branch.                 | Exact arithmetic control           |
| Valid finite derivation         | Change target, arity, rule, or active assumption; independent checker rejects.                      | Executed                           |
| Local block proof               | Reuse its assumption after the block; publication fails.                                            | Executed                           |
| Correct terminal assertion      | Add an unrelated unproved assertion; module fails despite the earlier success.                      | Executed                           |
| Correlated length pair          | Offer (1, 0) as a diagonal observation witness; no source refutation.                               | Paper theorem and arithmetic tests |
| Unchanged typed context         | Recycle identifiers after rebuild or alter a dependent index; require recheck or transport.         | Proposed integration test          |
| Fixed structure instance        | Change imports so another structure is selected; invalidate the sealed meaning or ask for a choice. | Proposed integration test          |

A shape-only Python model of the last two cases would not count as testing Lean's actual behavior. They must eventually be exercised in the real frontend.

### 17.5 A legitimate stopping outcome

If  $L_3$  does not improve authoring and diagnosis over  $L_2$ , retain the library, synthesis bridge, and editor obligation view. If  $L_4$  increases reader misunderstanding or maintenance cost relative to  $L_3$  and  $L_R$ , drop the separate mathematical syntax. If a primitive refinement type does not beat an optimized ordinary encoding on a documented bottleneck, leave the kernel unchanged.

These are not reasons to avoid the experiment. They identify what would count as learning from it. A reusable proof-producing contract library and a correct residual explorer are useful artifacts even if the best eventual language remains ordinary Lean.

## 18 Relationship to earlier type-theoretic work

Refinement-based synthesis is established research. Synquid uses polymorphic refinement types to guide synthesis and decompose specifications. Its tractable checking and synthesis fragment does not make arbitrary Lean propositions decidable. ALDER borrows the specification-directed viewpoint but makes the boundary of its automatic fragment and the retained Lean evidence explicit [13].

Explicit refinement systems distinguish logical specifications from computational contents, while work on logical-framework refinements relates refinement interpretation to proof irrelevance. These are relevant precedents for separating a carrier from its proved properties. They do not remove the need for source interpretation, witness scope, or a concrete translation into Lean [14, 15].

Dijkstra monads connect effectful computation to predicate transformers and verification conditions. ALDER’s residual mechanism is related in its backward flow of demands, but an unfinished mathematical justification is not itself an effectful program. The integration should reuse established program-verification semantics rather than invent an informal analogy in place of one [16, 12].

Definitional functoriality and Lean4Lean are relevant to the optional kernel research described in Section 15. Neither is required for the initial frontend. Existing Lean function-property automation is the immediate engineering comparison [17, 18, 11].

Finally, the nine round-three proposals supply most of the architectural baseline: same-object views, scope-closed support, separate structure selection, relational contracts, precision demands, and checked computation. ALDER’s distinct emphasis is the executable residual calculus, the realizable affine-frame interface, and a source-facing conditional-proof discipline. The article builds on the syntheses’ corrections instead of treating their convergence as independent experimental evidence [1, 2].

## 19 Conclusion

A concise proof language should not try to make mathematics rigorous by guessing more aggressively about what the author meant. It should preserve that meaning and make the omitted justification a tractable, inspectable interface.

ALDER proposes three main mechanisms. First, fixed mathematical objects acquire proved property views, while proof-producing steps carry explicit residual telescopes until their obligations are discharged. Second, a finite grounded fragment computes alternative sufficient premise sets with proof DAGs, and reports its boundary rather than confusing incomplete search with falsity. Third, behavioral synthesis can use finite realizable observation frames when a source denotation theorem and an affine coverage theorem justify them.

The exact-quotient, local-differentiation, and algebraic-root examples show why these mechanisms matter: the missing evidence is often routine, but its type distinguishes a correct proof from a subtly different claim. The executable companion tests the central finite inference and authoring path. The remaining task is a small Lean implementation and an honest comparison with equally equipped ordinary Lean, not another expansion of the feature catalogue.

## A Answers to the two sets of fourth-round questions

The labels A1–A10 follow *From Properties to Proof Obligations*; B1–B10 follow *Nine Refinements*. The entries summarize an answer or explicitly identify work not executed here.

| Id  | Question                              | Alder’s answer and evidence boundary                                                                                                                                                                     |
|-----|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A1  | First use-site interface?             | One exact-quotient property-implicit binder and method application; compare with the same helper in Lean. Alder-0 parses only its abstract propositional support model, not the real binder.             |
| A2  | Finite automatic fragment?            | Finite typed arena, bounded first-order theorem instantiation, then Horn antichain closure. Completeness is relative to the materialized rules. Sections 6–7.                                            |
| A3  | What data may inference choose?       | Only witness holes explicitly delegated under a fixed specification. Structures, representations, and statement-determining holes are fixed or exposed as ambiguity. Section 4.                          |
| A4  | Required methods?                     | Restrict helper context and allowed registry before inference. A finite derivation-policy automaton is a possible extension; not implemented. Section 6.                                                 |
| A5  | First behavioral bridge?              | The list-expression affine model with a realizable affine frame. Paper proofs and exact tests here; Lean normalization and source reification remain to be linked. Section 11.                           |
| A6  | Negative Leant selection?             | A concrete admissible counterexample plus source transfer, or an exact proved negation. Unrealisable and merely model-relative witnesses do not refute the candidate.                                    |
| A7  | First nonalgebraic transfer?          | A held-out local-analysis task selected after the interface is frozen. The derivative theorem inspected here is development material, not a held-out result.                                             |
| A8  | Retained/rechecked evidence?          | Exact target, proof, context, source correspondence, structures, provider theorem dependencies, and policy. Rebuild local indices first; typed context embeddings are a later optimization. Section 14.  |
| A9  | Performance gate for larger checkers? | Instrument a real file and compare against existing tactics under a chosen trust policy. The fast Python experiment is not that result.                                                                  |
| A10 | When end the separate language?       | Keep only library/services/editor views if the property or narrative layer gives no advantage against matched controls. Section 17.                                                                      |
| B1  | Evidence-index cost on a real file?   | Not measured here. The Python result reports finite-graph counts only; production metrics are specified in Section 17.                                                                                   |
| B2  | Which rules actually fire?            | The executable records aggregate rule-combination work. There is no measured Mathlib rule histogram or comparison with single-lemma lookup yet.                                                          |
| B3  | Does simplification break anchors?    | Reuse by current definitional checking or retain an explicit equality/iff transport edge. Never rekey by printed strings. Section 14.                                                                    |
| B4  | One reifier for many checkers?        | Share a typed expression grammar and denotation interface where semantics genuinely agree. Each consumer still proves its own correctness and original-target bridge. The prototype has no Lean reifier. |
| B5  | Completeness of Leant abstractions?   | Theorem 11.2 gives a concrete completeness criterion for affine equality on a covered realizable domain. It makes no blanket claim about every Length contract.                                          |

| Id  | Question                                   | Alder's answer and evidence boundary                                                                                                                              |
|-----|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| B6  | Almost-everywhere consumers?               | Register exact integral congruence and other consumer theorems with fixed measure arguments. A pointwise upgrade can use continuity and full support. Section 12. |
| B7  | Do lexical contexts shorten FLT preambles? | Unmeasured; dictionary selection is not assumed to solve all instance-search conflicts. Preserve the real file and test it.                                       |
| B8  | Bounded rule generation?                   | Section 7 gives a finite arena, explicit theorem-shape restriction, and the $t^k$ grounding bound. The prototype starts from already-ground rules.                |
| B9  | What does a compact reader recover?        | Specify a blinded recovery test for domain, measure, branch, precision, and quantifiers. No human reading study was run here.                                     |
| B10 | Independent implementations?               | The Horn contract and supplied oracle tests can be implemented independently. The separate Python checker is useful but is not a second production Lean frontend. |

## B Package contents and reproduction

The archive contains the self-contained `Alder.tex`, its rendered PDF, the companion sources and authored examples, generated Lean and JSON traces, a source register, test results, build instructions, and file hashes. No font files or vendored dependencies are included.

From the package root, run:

```
python companion/test_alder.py
python companion/alder_core.py companion/examples/quotient.alder \
  --lean companion/generated/Quotient.lean \
  --events companion/generated/quotient.json
latexmk -pdf -interaction=nonstopmode -halt-on-error Alder.tex
```

Python 3.10 or newer is required by the companion. The retained run used 3.13.5. The test script writes a new result JSON including its actual runtime and current source hashes; runtime is not expected to be byte-reproducible across machines.

The generated Lean files can additionally be checked with an installed Lean toolchain:

```
lean companion/generated/Quotient.lean
lean companion/generated/Locality.lean
```

Those commands are reproduction instructions, not a record that they were executed here. The generated theorems use only basic propositions and implication structure. Real integration still needs instantiated mathematical theorem registrations and an exact source-target bridge.



## References

- [1] Codex. *From Properties to Proof Obligations: A Critical Synthesis of the Nine Round-3 Proposals*. Revised 6 September 2026. Brainstorm, revision `bf3333905995060870f8585e297ffc16f3fe7e86`. Main report and its convergence crosswalk.  
<https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/synthesis/unified-report.tex>.
- [2] Claude (Anthropic). *Nine Refinements: Property-Aware Typing in a Mathematician's Proof Language over Lean*. Revised 6 September 2026. Same Brainstorm revision; main report and `appendix.tex`.  
[https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/unified\\_report/unified\\_report.tex](https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/unified_report/unified_report.tex).
- [3] V. Reshetnikov and contributors. `AutonomousIteratedDeriv.lean`. ProveIt, revision `24ce8bd743eaab64a91ce90725ea00f498d319d2`. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [4] V. Reshetnikov and contributors. `AlgebraicInverseGerm.lean`. Same ProveIt revision.  
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AlgebraicInverseGerm.lean>.
- [5] Anthropic and credited project contributors. `S_FreyPackage_freyCurveInt_map.lean`. *Fermat's Last Theorem in Lean*, revision `aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`.  
[https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S\\_FreyPackage\\_freyCurveInt\\_map.lean](https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_freyCurveInt_map.lean).
- [6] Anthropic and credited project contributors. `S_Algebra_exists_algHom_equiv_pi.lean`. Same Fermat repository revision. [https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S\\_Algebra\\_exists\\_algHom\\_equiv\\_pi.lean](https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean).
- [7] V. Reshetnikov and contributors. `src/Leant/Synth/Verification.hs`. Leant, revision `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs>.
- [8] V. Reshetnikov and contributors. `src/Leant/Synth/PostVerification.hs`. Same Leant revision.  
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/PostVerification.hs>.
- [9] V. Reshetnikov and contributors. `src/Leant/Synth/Length/Contract.hs`. Same Leant revision.  
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Length/Contract.hs>.
- [10] Lean team. *The Lean Language Reference*, chapters on the type system, propositions, functions, and proof validation. Accessed 6 September 2026.  
<https://lean-lang.org/doc/reference/latest/The-Type-System/>.
- [11] The mathlib community. *Mathlib.Tactic.FunProp*: function-property theorem registration, decomposition, and transition rules. Accessed 6 September 2026.  
[https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Tactic/FunProp.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html).
- [12] Lean team. *The Lean Language Reference: The mvcgen Tactic*, overview of predicate-transformer semantics and verification conditions. Accessed 6 September 2026.  
<https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/>.
- [13] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. PLDI 2016. Preprint v3, 21 April 2016. <https://arxiv.org/abs/1510.08419v3>.
- [14] J. E. Ghalayini and N. Krishnaswami. Explicit Refinement Types. 2023.  
<https://arxiv.org/abs/2311.13995>.

- [15] W. Lovas and F. Pfenning. Refinement Types for Logical Frameworks and Their Interpretation as Proof Irrelevance. *Logical Methods in Computer Science* 6(4), 2010.  
<https://arxiv.org/abs/1009.1861>.
- [16] D. Ahman, C. Hrițcu, K. Maillard, G. Martínez, G. Plotkin, J. Protzenko, A. Rastogi, and N. Swamy. Dijkstra Monads for Free. POPL 2017; extended preprint v3, 2019.  
<https://arxiv.org/abs/1608.06499v3>.
- [17] T. Laurent, M. Lennon-Bertrand, and K. Maillard. Definitional Functoriality for Dependent (Sub)Types. ESOP 2024; extended version v2. <https://arxiv.org/abs/2310.14929v2>.
- [18] M. Carneiro. Lean4Lean: Verifying a Typechecker for Lean, in Lean. Preprint v3, 14 September 2025.  
<https://arxiv.org/abs/2403.14064v3>.